

Undirected Graph $G=(V,E)$ (or $G=(N,A)$)

V : Set of vertices (nodes...)

Edge $\{a,b\}$ is an unordered pair of nodes

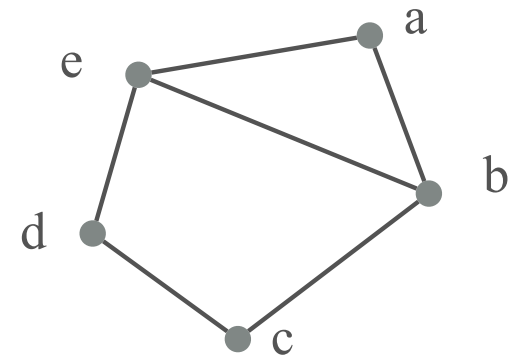
$E \subset V \times V$: Set of vertices (arcs...)

Undirected graph: $G=(V,E)$

- Typically, don't allow $\{a,a\}$ in E

Vertex Set: $V = (a,b,c,d,e)$

Edge Set: $E = (\{a,b\}, \{b,c\}, \{c,d\}, \{d,e\}, \{e,a\}, \{e,b\})$



degree = 3

Directed Graph $G=(V,E)$

V is the set of vertices

An edge $e = (a,b)$ is an ordered pair from a to b , where a, b in V

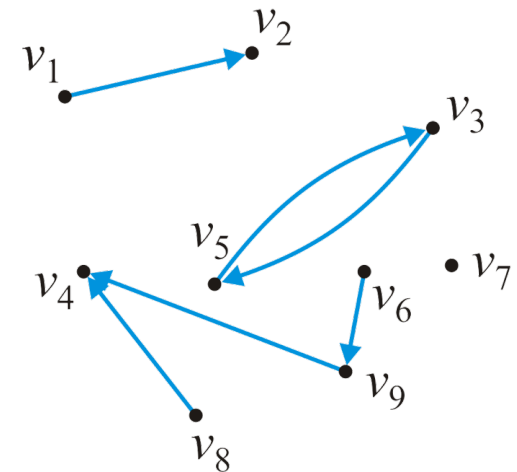
Note: edge (b, a) is different from edge (a, b)

Example: streets, ...

Max. no. edges = $|V|^2$ (allow self loops)

Out-degree of a vertex: no. edges leaving vertex

In-degree of a vertex: no. edges entering vertex



$$E = \{(v_1, v_2), (v_3, v_5), (v_5, v_3), (v_6, v_9), (v_8, v_4), (v_9, v_4)\}$$

Breadth-First Search: Uses a Queue

1. Mark all vertices as unvisited, parents as NULL, depth as -1
 2. Choose any unvisited vertex, mark it as **visited** and enqueue it onto queue
 3. While the queue is not empty:
 - Dequeue top vertex v from the queue. Do work to be done on that vertex
 - If $\text{parent}[v] == \text{NULL}$, set depth to 0; otherwise, set depth to $\text{depth}[\text{parent}[v]] + 1$
 - For each vertex adjacent to v (e.g. in out list) that has not been visited: Mark it visited, mark its parent as v , and enqueue it
 - Mark v as done
 4. If there are unvisited vertices, choose any unvisited vertex, mark it as **visited**, enqueue it and repeat step 3
- This can handle graphs that are not connected
 - Marking as visited avoids cycles
 - Complexity: $O(\#V + \#E)$, reduces to $O(\#E)$ if strongly connected
 - Size of queue is $O(\#V)$

Depth-First Search

Recursive implementation:

1. Mark all vertices as unvisited; mark all parents as NULL
2. While there are vertices marked as unvisited:
 - Select unvisited vertex v , mark as visited:
 - Do DFS(vertex)

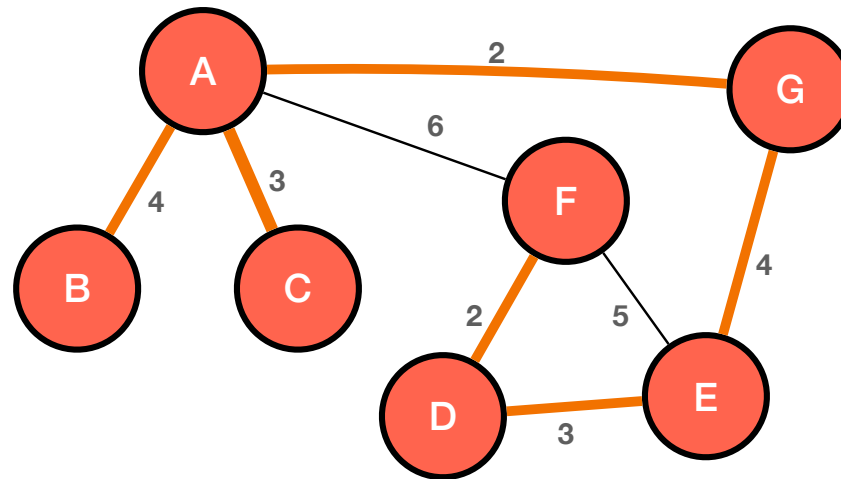
DFS(vertex):

- For neighbors of vertex
 - If neighbor is unvisited, mark as visited and do DFS(neighbor)
- This can handle graphs that are not strongly connected
 - Marking as visited avoids cycles
 - Complexity: $O(\#V + \#E)$, reduces to $O(\#E)$ if strongly connected
 - Size of queue is $O(\#V)$

Minimum Spanning Trees

- A **minimum spanning tree** (MST) of an undirected weighted graph (V, E) with weights $w(\cdot)$:

Connected subgraph (V, E') which is a tree and for which $\sum_{\{i,j\} \in E'} w(i,j)$ is minimized



Kruskal's Algorithm

- Sort edges by weight in ascending order
- Start with empty set T (note: it is promising)
- For each edge e in sorted list
 - If adding edge e to T does not create cycle in $(V, T \cup e)$
 - ...add it to MST: $T = T \cup \{e\}$
 - Claim: T is now promising set with one more edge
- Stop when you have $\#V - 1$ edges in T

Different MST Algorithm: Prim-Jarnik

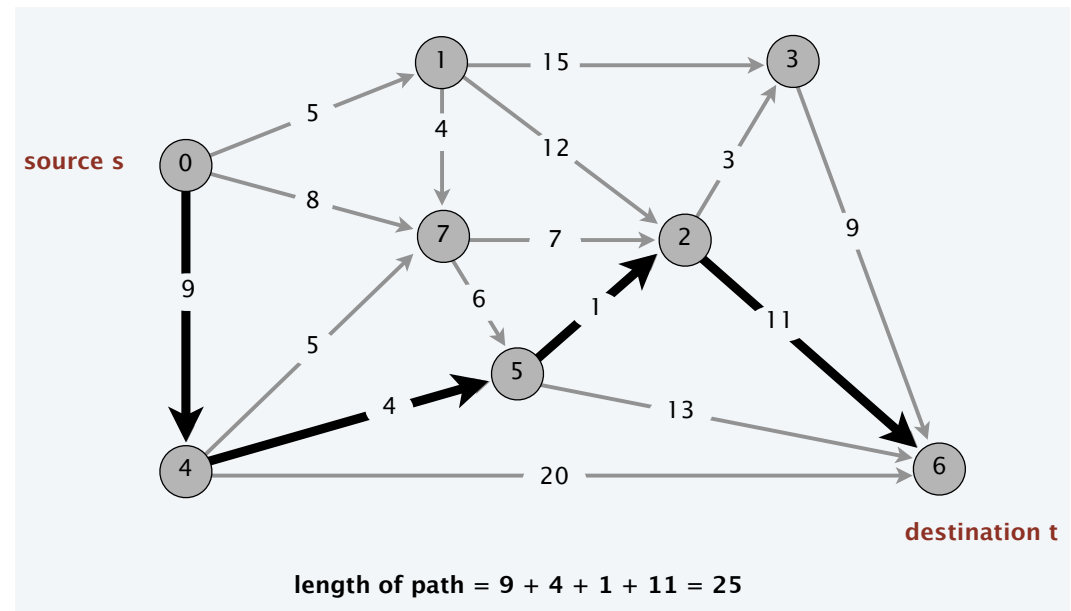
- Traverse $G = (V, E)$ starting at any node
 - Maintain priority queue of nodes (e.g. binary heap, Fibonacci heap)
 - set priority to weight of the cheapest edge that connects them to MST
- Un-added nodes start with priority ∞
- At each step
 - Add the node with lowest cost to MST
 - Update (“relax”) neighbors as necessary
- Stop when all nodes added to MST

Borůvka's Algorithm

- Earliest MST algorithm: 1926. Application: design of power grid
- For **every** connected set in a forest,
 - Select smallest weight edge that leaves connected set
 - Add it to the MST
 - But don't add it twice if same edge selected by two connected sets
- In principle, merges at least half of the trees at each time: $O(\log(n))$ iterations
 - Easy to parallelize
- Each pass is $O(\#E)$

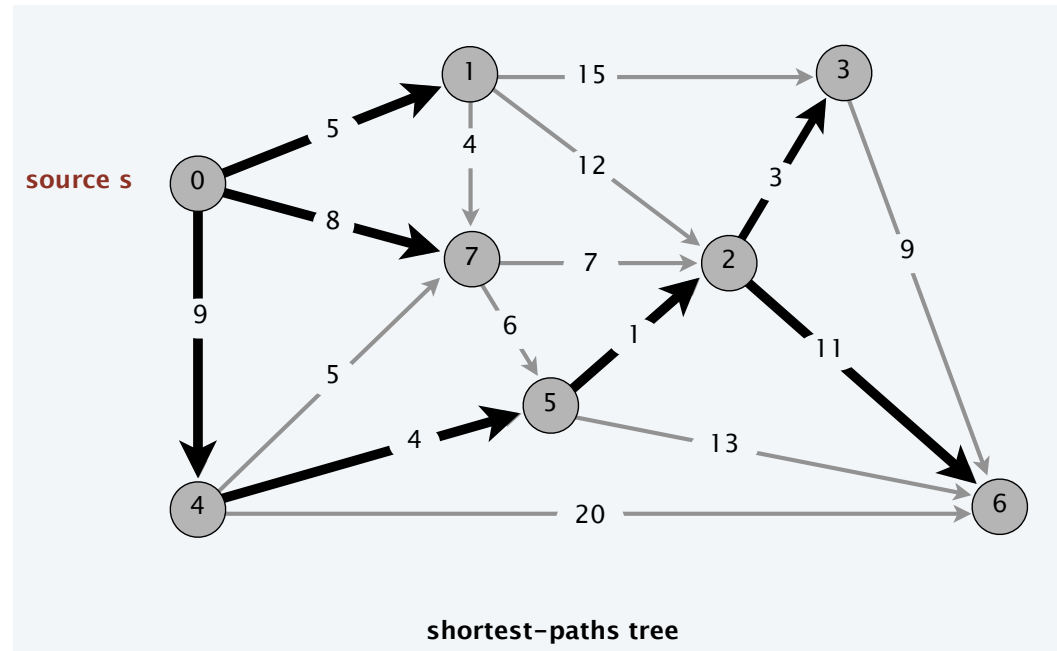
What is a Shortest Path?

- Given weighted, directed graph $G(V,E)$ (weights on edges)...
 - ...what is shortest path from vertex s to t ?
 - Distance is additive
- Applications
 - Google maps
 - Routing packets on the Internet
 - Social networks



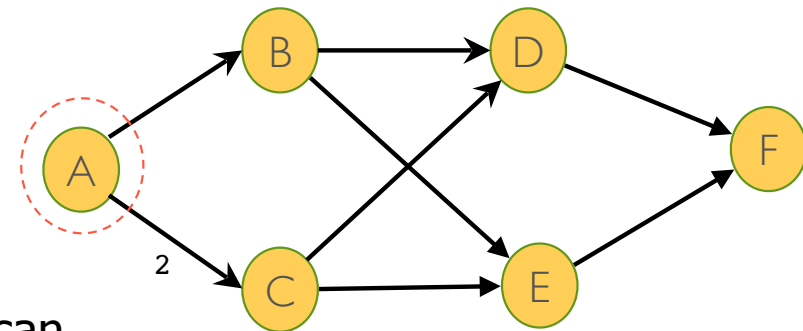
Single Source Shortest Paths (SSSP)

- Given a graph and a source vertex
 - find the shortest paths to all other vertices
 - results in a shortest-path tree
 - Single directed path to every other vertex



Important Property of Shortest Paths

- **Lemma:** The shortest path from vertex s to a vertex t is composed of shortest paths to and from any intermediate vertices
 - Proof by contradiction on path from s to t
 - Consider intermediate vertex v
 - If better path from s to v was possible, then can replace original path to v and get shorter
 - If better path from v to t was possible, then can original path from v to t and get shorter
 - Hence, original path was not shortest path
- **Bellman's Principle of Optimality**

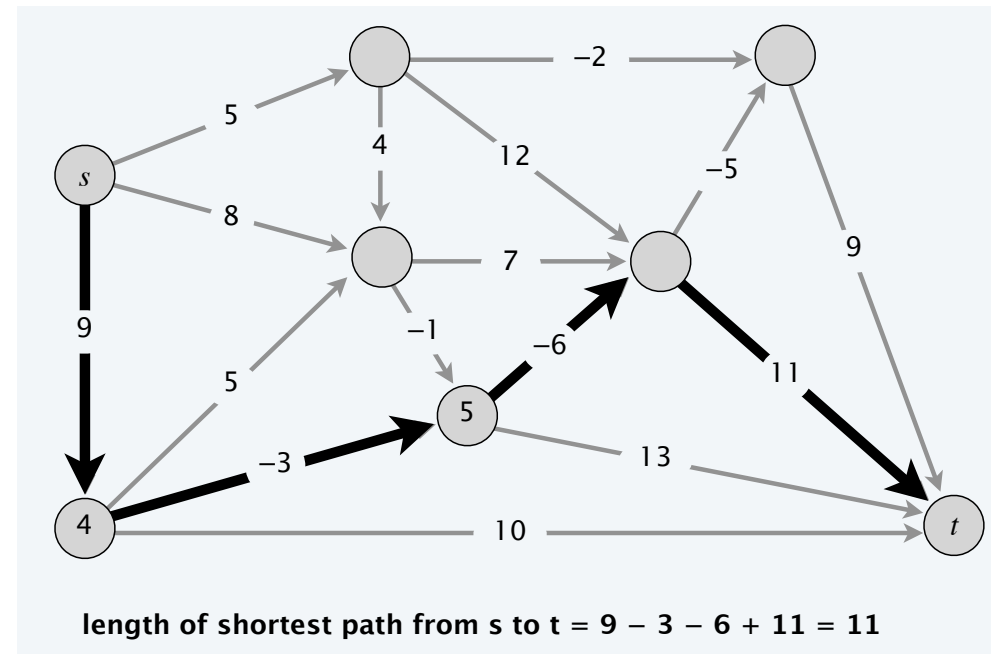


Dijkstra's Algorithm

- Greedy Algorithm: Assumes all edges have **nonnegative** weights
 - Maintain a set of **explored** nodes S for which algorithm has determined $D[u]$ = length of a shortest s to u path
 - Initialize $S = \{s\}$, $D[s] = 0$; $D[v] = \text{infinity}$
 - Choose unexplored node $v \notin S$ which minimizes $D[u] + w(u, v)$, $u \in S$
Set $D[v] = \min\{\min_{\{(u,v) \in E: u \in S\}} D[u] + w(u, v), D(v)\}$, and add v to S
Set $\text{pred}[v] = \text{vertex } u \text{ in } S \text{ that achieves } d[v]$
 - Repeat until all vertices are explored, so $S = V$
 - Path to any vertex can be found by using $\text{pred}[]$ labels

Shortest paths: Graphs with Negative Weights

- Need different algorithm
- Dijkstra's algorithm: "label-setting"
 - Once vertex exits priority queue, its distance label never changes
- Different class of algorithms: "label-correcting"
 - Exploit Bellman's principle of optimality
 - Will handle some cases with negative weights



Bellman-Ford Algorithm

- **Notation:** $D(k,v)$ is shortest distance from vertex s to vertex v on paths with less than or equal to k edges

Initial value: $D(0,v) = \infty, v \neq s; 0, v = s$

Bellman equation $D(k, v) = \min \left\{ D(k-1, v), \min_{(u,v) \in E} [D(k-1, u) + w(u, v)] \right\}$ (DP)

- Bellman-Ford Algorithm
 - For each $v \in V$, set $D(0,v) = \infty, v \neq s; 0, v = s$; set $\text{pred}[v] = \text{null}$
 - For $k = 1$ to $|V| - 1$:
 - For each $v \in V$, set $D(k, v) = D(k-1, v)$
 - For each $(u, v) \in E$:
 - if $D(k, v) > D(k-1, u) + w(u, v)$:
 - Set $D(k, v) = D(k-1, u) + w(u, v)$, set $\text{pred}[v] = u$

Better Bellman-Ford Implementation

- Use a queue of vertices where distances have changed
 - Avoid looking at edges out of vertices where distances have not changed
- Algorithm
 - Initialize $D[s] = 0$; $D[v] = \infty, v \neq s$; $\text{pred}[v] = \text{null}$
 - Insert s into queue Q ; mark $\text{inqueue}[s] = \text{true}$, mark $\text{inqueue}[v] = \text{false}, v \neq s$
 - While Q is not empty:
 - Select u out of queue, mark $\text{inqueue}[u] = \text{false}$
 - For each edge (u,v) in E :
 - If $D[v] > D[u] + w(u, v)$:
 - Set $D[v] = D[u] + w(u, v)$, set $\text{pred}[v] = u$
 - if $\text{inqueue}[v] = \text{false}$, add v to Q , mark $\text{inqueue}[v] = \text{true}$

New Variation: All Pairs Shortest Paths

Input: Directed graph $G = (V, E)$, where $V = \{1, 2, \dots, n\}$, with edge-weight function $w : E \rightarrow \mathbb{R}$

- Weights may be negative, but no negative cycles

Output: $n \times n$ matrix of shortest-path lengths $D(i, j)$ for all $i, j \in V$ (and routes)

Possible solution:

- Run Bellman-Ford once from each vertex as starting node
- Time $O(|V|^2 E)$
- Dense graph ($\Theta(n^2)$ edges) $\Rightarrow O(n^4)$ time in the worst case

Floyd-Warshall Algorithm

- Initialize $n \times n$ weighted adjacency matrix A and initial distance matrix $D^{(0)}$ as

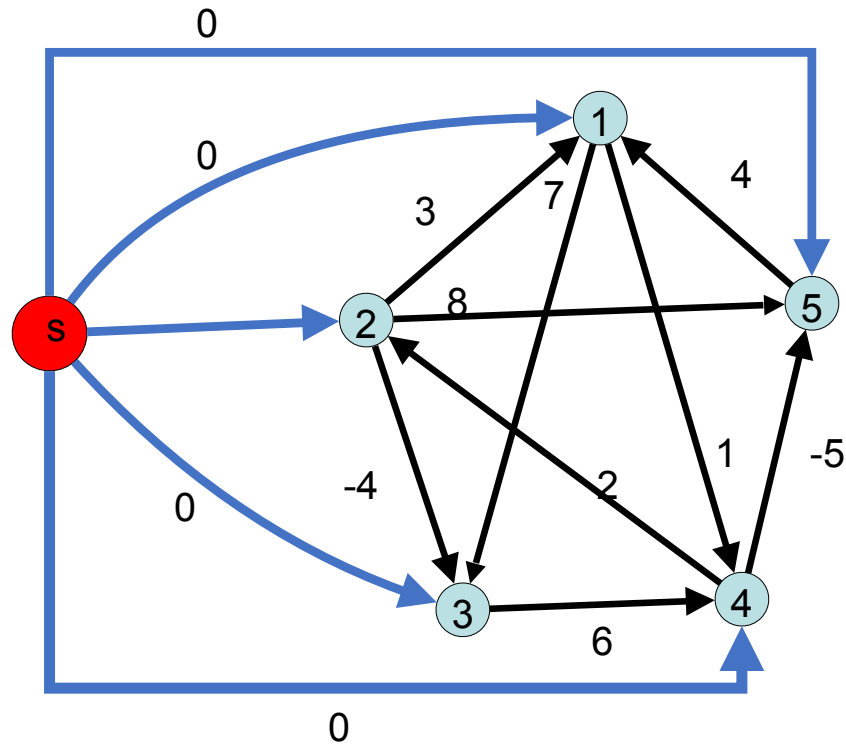
$$A(i, j) = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } (i, j) \in E, \\ \infty & \text{if } (i, j) \notin E \end{cases} \quad D^{(0)}(i, j) = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } i \neq j \end{cases}$$

- Note that $D^{(0)}$ is the matrix of shortest distances among paths with zero edges!
- A simple algorithm
 - For $m = 1, 2, \dots, n-1$:

$$D^{(m)}(i, j) = \min_{k \in V} [D^{(m-1)}(i, k) + A(k, j)]$$

- Note that $A(j, j) = 0$, so that $D^{(m)}(i, j) = D^{(m-1)}(i, j)$ is possible, and thus $D^{(m)}(i, j)$ is non-increasing

Johnson's algorithm



Main idea: Add a dummy vertex with a directed edge of weight 0 to all other vertices

Undirected Graph $G=(V,E)$ (or $G=(N,A)$)

V : Set of vertices (nodes...)

Edge $\{a,b\}$ is an unordered pair of nodes

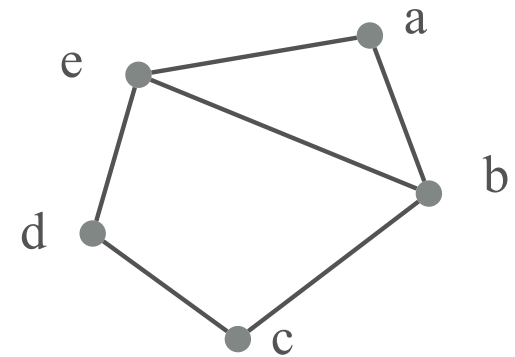
$E \subset V \times V$: Set of vertices (arcs...)

Undirected graph: $G=(V,E)$

- Typically, don't allow $\{a,a\}$ in E

Vertex Set: $V = (a,b,c,d,e)$

Edge Set: $E = (\{a,b\}, \{b,c\}, \{c,d\}, \{d,e\}, \{e,a\}, \{e,b\})$



degree = 3

Problem: Shortest Path from s to a single t

- We only want a path to a destination
 - But all algorithms compute paths from s to every destination
 - When can we stop?
 - Can we focus on getting only to where we want to go?
 - This is the Google maps or Waymo problem
- New algorithm: A^* search
 - Will search in direction of t for shortest path
 - A modification of Dijkstra to bias the search
 - Guaranteed optimal under certain conditions

Heuristic Distance Estimates

- Let $h : V \rightarrow \mathbb{R}$ be a heuristic function that approximates the distance from vertex v to target t
- We call h **admissible** if: for each vertex v : $h(v) \leq D(v,t)$, where $D(v,t)$ is the true shortest path distance
- We call h **consistent** if: For each (u,v) in E : $h(u) \leq w(u,v) + h(v)$.
 - For example, Euclidean distance to target on the plane
 - Euclidean distance is admissible and consistent

A* Search

- Simple idea: use heuristic $h()$ to reweigh edges
 - $w_h(u, v) = w(u, v) - h(u) + h(v)$
 - Measures deviation from ideal distance estimate by h
 - Consistent heuristics guarantee non-negative weights
- Solve using Dijkstra's algorithm
- Result: If heuristic is consistent and admissible, we get an optimal shortest path
 - Dijkstra's distance is biased towards vertices that have shorter distance estimates

Algorithm Description

- Keep track of best distance from s , but use as key in the priority queue the optimistic distance estimate to destination t
- Initially, mark distances $D[s] = 0$, $D[v] = \infty$, $v \neq s$, and $\text{key}[v] = D[v] + h(v,t)$, $\text{pred}[v] = \text{NULL}$. Insert into priority queue Q .
- While t is still in Q :
 - Remove_min from Q , as vertex u
 - If $u = t$, break. We have the path we want.
 - For all v such that $(u, v) \in E$, and v is still in Q , compute $D_v = D[u] + w(u, v)$.
 - If $D_v < D[v]$, update $D[v] = D_v$, $\text{pred}[v] = u$, $\text{key}[v] = D_v + h(v,t)$, and Decrease_key of v in Q
- Note: We don't have to keep track of $\text{key}[v]$ explicitly in Q if we know $h(v,t)$

Maximum Flow in Flow Networks

- A feasible st-flow (flow) $f : E \rightarrow \mathbb{R}^+$ is a function that satisfies

$0 \leq f(u, v) \leq c(u, v)$ for all edges $(u, v) \in E$ (capacity satisfied)

For every $v \in V - \{s, t\}$, flow is conserved: $\sum_{(u,v) \in E} f(u, v) = \sum_{(v,w) \in E} f(v, w)$

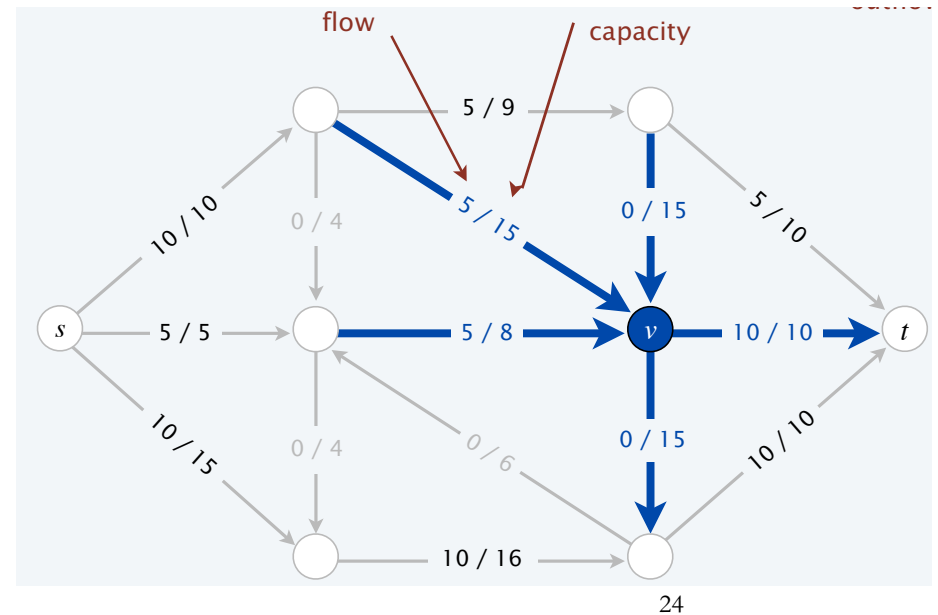
- Value of flow: net flow out of s

$$val(f) = \sum_{(s,v) \in E} f(s, v) - \sum_{(v,s) \in E} f(v, s)$$

Value is 25 on the right...

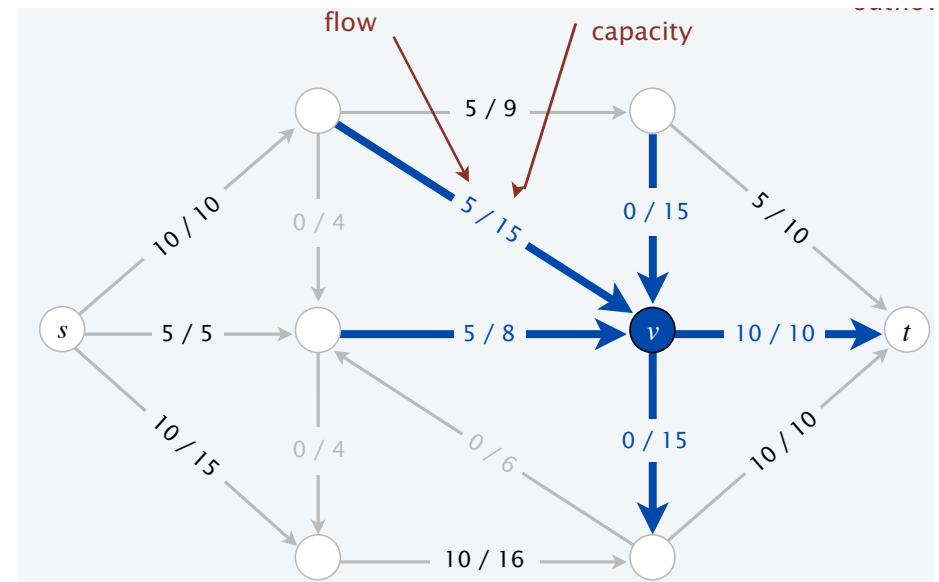
Also net flow into t

- Maximum flow problem: find flow f that maximizes $val(f)$



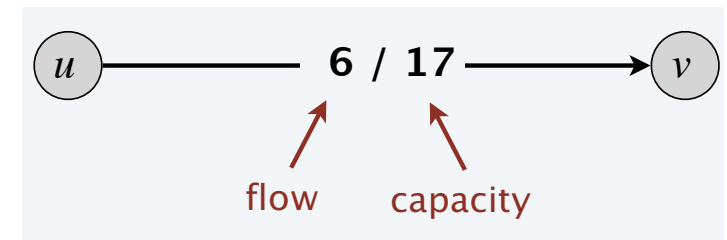
Max-flow, Min-cut are Related

- Formally, they are dual of each other
- Result to come: optimal value of max-flow problem is the same as optimal value of min-cut problem
- Any s-t cut is an **upper bound** to the max flow problem
- Any feasible flow provides a **lower bound** to the min-cut problem



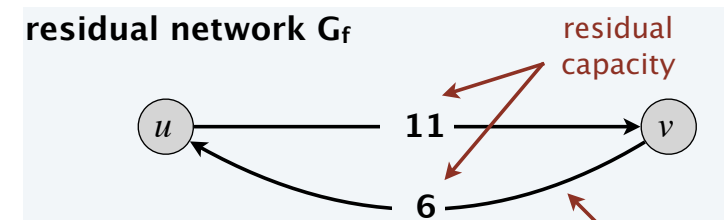
Residual Network

- Key concept: Residual network given a flow f
- Original edge. $e = (u, v) \in E$
 - Flow $f(u, v) > 0$, capacity $c(u, v)$
- Want to create a way to “undo” assigned flow
 - Create a reverse edge (v, u) with capacity $c(v, u) = f(u, v)$
 - Flow on this reverse edge reverses the existing flow $f(u, v)$



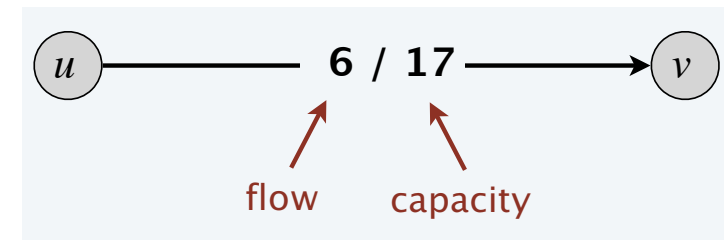
- Residual network $G_f = (V, E_f)$
 - Capacity on residual network edges:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & (u, v) \in E \\ f(v, u) & (v, u) \in E \end{cases}$$
- $E_f = \{(u, v) \in E : f(u, v) < c(u, v)\} \cup \{(v, u) : (u, v) \in E, f(u, v) > 0\}$



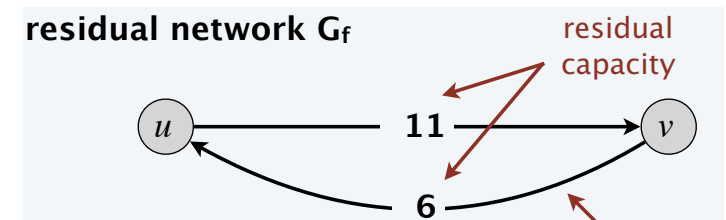
Residual Network

- Key concept: Residual network given a flow f
- Original edge. $e = (u, v) \in E$
 - Flow $f(u, v) > 0$, capacity $c(u, v)$
- Want to create a way to “backtrack” flow
 - Create a reverse edge (v, u) with capacity $c(v, u) = f(u, v)$
 - Flow on this reverse edge reverses the existing flow $f(u, v)$



- Residual network $G_f = (V, E_f)$
 - Capacity on residual network edges:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & (u, v) \in E \\ f(v, u) & (v, u) \in E \end{cases}$$
- $E_f = \{(u, v) \in E : f(u, v) < c(u, v)\} \cup \{(v, u) : (u, v) \in E, f(u, v) > 0\}$



Augmenting Paths

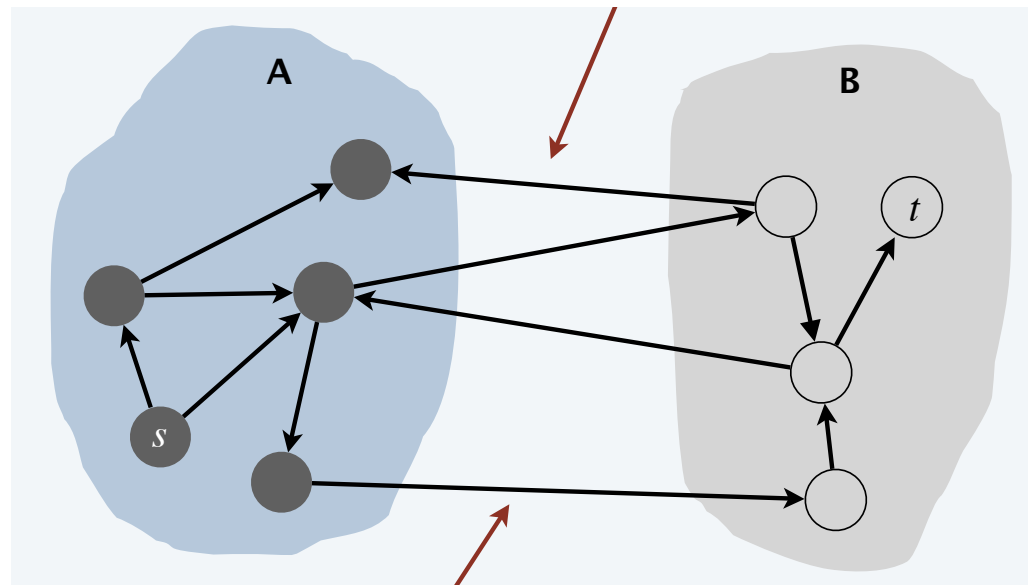
- An augmenting path is a simple $s \rightarrow t$ path in the residual network G_f
- The bottleneck capacity $bottleneck(P, G_f)$ of an augmenting path P is the minimum residual capacity of any edge in P
- For bottleneck capacity δ , we can augment flow on P to modify f :
 - If $(u, v) \in P, (u, v) \in E$, augment $f'(u, v) = f(u, v) + \delta$
 - If $(v, u) \in P, (u, v) \in E$, this is a reverse edge; reduce $f'(u, v) = f(u, v) - \delta$
- **Key property.** Let f be a flow and let P be an augmenting path in G_f . Let δ be the flow on P (bottleneck capacity). Then, the augmented f' is a feasible flow in the original graph, and $val(f') = val(f) + \delta$
- An augmenting path is a **negative-cost cycle with capacity!**

Ford-Fulkerson Algorithm ('56)

- Start with flow $f(u,v) = 0$, $(u, v) \in E$. Form the residual network $G_f = G$
- While there exists an $s \rightarrow t$ path P in the residual network
 - Compute residual capacity δ on P , and augment flow f using flow δ on path P
 - Update residual network using new flow f , as G_f
- When no path can be found, return flow f
- Properties of algorithm:
 - At each iteration, flow increases
 - If capacities are integers, this terminates: flow out of s increases by at least 1 at each iteration, and has upper limit as the sum of capacities of edges leaving s

Max-Cut, Min Flow

- Max-flow min-cut theorem: Value of a max flow = capacity of a min cut
- Augmenting path theorem: A flow f is a max flow if and only if there exists no augmenting paths in its residual graph
- At optimality, edges from B to A must have 0 flow
- At optimality, edges from A to B must have flow equal to capacity
- Can compute a min cut from a max flow by letting A = nodes reachable from s in residual graph of f



Edmonds-Karp ('72)

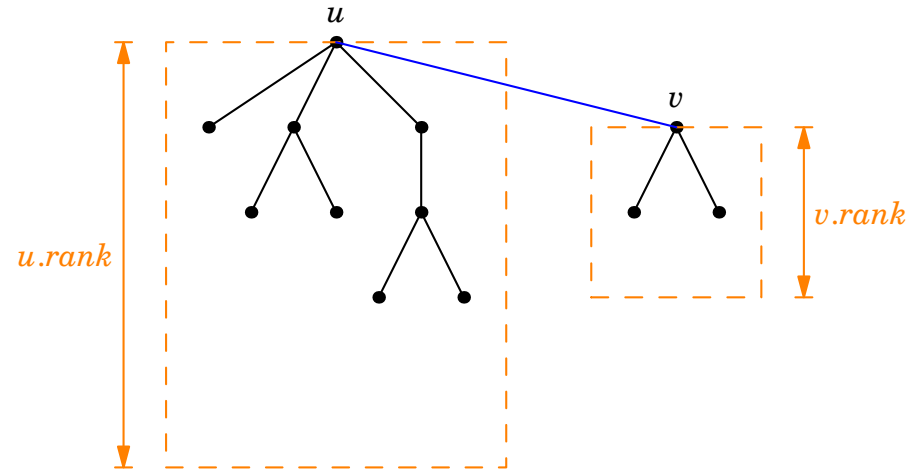
- Choose augmenting paths in residual network that have least number of edges
 - Use BFS to find augmenting paths!
- Nice properties:
 - Length of augmenting path does not decrease in iterations
 - After at most $|E|$ iterations, length of augmenting path must have increased
 - Maximum length of augmenting path is $|V|-1$
 - Bound on number of augmenting paths: $O(|V||E|)$
 - Bound on complexity: $O(|V||E|^2)$
- Another approach: choose augmentations with sufficient capacity
 - Capacity scaling used in best algorithms today

Disjoint Sets

- Complexity
 - Height update on merge:
 - h_1 is height of taller tree, h_2 is height of shorter tree
 - If $h_1 > h_2$, height of h_1 does not change
 - If $h_1 = h_2$, h_1 increases by 1
 - Find complexity: time to find root of tree = height of tree: $O(\log(n))$
 - Merge complexity: compare heights, point shorter root to taller one: $\Theta(1)$
 - Worst case complexity of union-find: $O(\log(n))$
(too slow!)

Updating Ranks

- **Rank** is an **upper bound** on height
 - Height is hard to keep track of exactly because of path compression
- How is rank assigned?
 - Rank of single nodes = 0
 - If 2 trees of equal ranks are merged, $\text{rank}(\text{new root}) = \text{rank}(\text{trees}) + 1$
 - Without path compression, this makes $\text{rank} = \text{height}$
 - After path compression, rank is upper bound to height
- **Ranks only change for root when adding children**
 - Don't update ranks of non-roots because only roots matter!

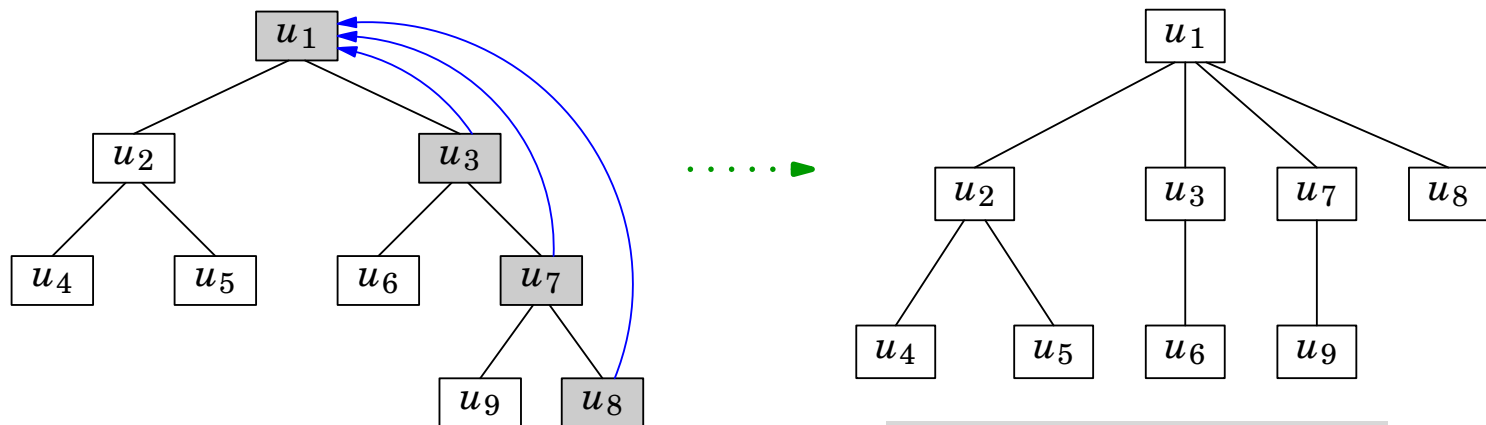


Algorithm: UNION($\tilde{u}, \tilde{v}$)

```
1  $u \leftarrow \text{FIND-SET}(\tilde{u})$ 
2  $v \leftarrow \text{FIND-SET}(\tilde{v})$ 
3 if  $u.rank = v.rank$  then
4      $u.rank \leftarrow u.rank + 1$ 
5      $v.parent \leftarrow u$ 
6 else if  $u.rank > v.rank$  then
7      $v.parent \leftarrow u$ 
8 else
9      $u.parent \leftarrow v$ 
```

More on Rank and Path Compression

- Path compression illustration: Find u_8



- Rank does not change!
 - But height dropped by 1

Algorithm: FIND-SET(u)

```
1  $A \leftarrow \emptyset$ 
2  $n \leftarrow u$ 
3 while  $n$  is not the root do
4    $A \leftarrow A \cup \{n\}$ 
5    $n \leftarrow n.parent$ 
6 for each  $x \in A$  do
7    $x.parent \leftarrow n$ 
8 return  $n$ 
```

Disjoint Sets

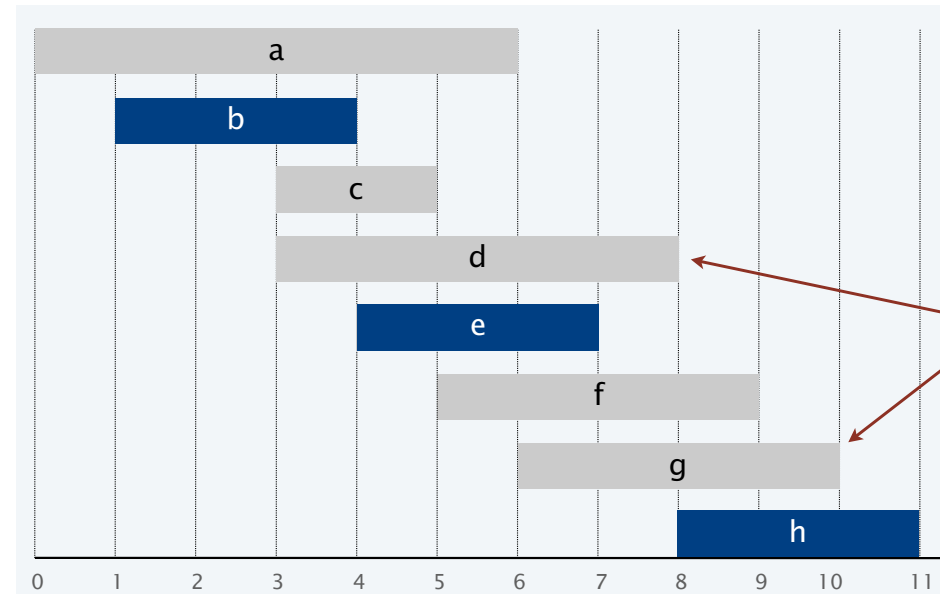
- **Theorem:** Starting from an empty data structure, any sequence of M union and find operations on N objects takes $O(N + M \log^*(N))$ time
 - $\log^*(N)$: Ackerman function
 - $A(i) = 2^{A(i-1)}$; $A(0) = 1$
 - $A(1) = 2$, $A(2) = 4$, $A(3) = 16$, $A(4) = 2^{16} = 65536$, $A(5) = \dots$,
 - $A(6) = \text{VERY VERY VERY BIG!}$
 - Inverse Ackerman: $i = \log^*(N)$
 - $\log^*(N) = \text{min number times you take } \log_2(\log_2(\dots)) \text{ to get smaller than or equal to } 1$
 - For all practical purposes, it is $O(1)$ (Ch. 21, CLRS)
 - Disjoint set complexity per union-find: almost $O(1)$

Greedy Algorithms (Ch. 16, CLSR v3)

- Greedy algorithm: an algorithm that builds a solution adding an element at a time that is the locally optimal choice at that time
 - Uses a simple rule to select next element
 - No backtracking (!!!) so algorithms are fast (near-linear time)
 - Greedy algorithms are not always optimal for problems. When not optimal, may provide fast approximate algorithms...
 - Special classes of problems can be solved to optimality by greedy algorithms
 - Proving optimality is challenging

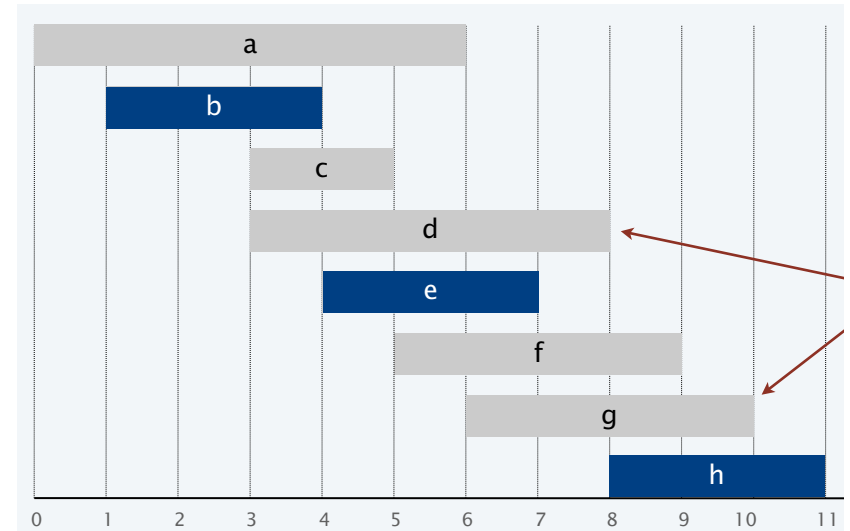
Interval Scheduling

- Jobs $\{1, 2, \dots, N\}$ occur at fixed times, with start/finish times $s(j)$, $f(j)$ for job j
- Single processor
- 2 jobs are compatible if they don't overlap
- Goal: Maximize number of jobs scheduled that are compatible



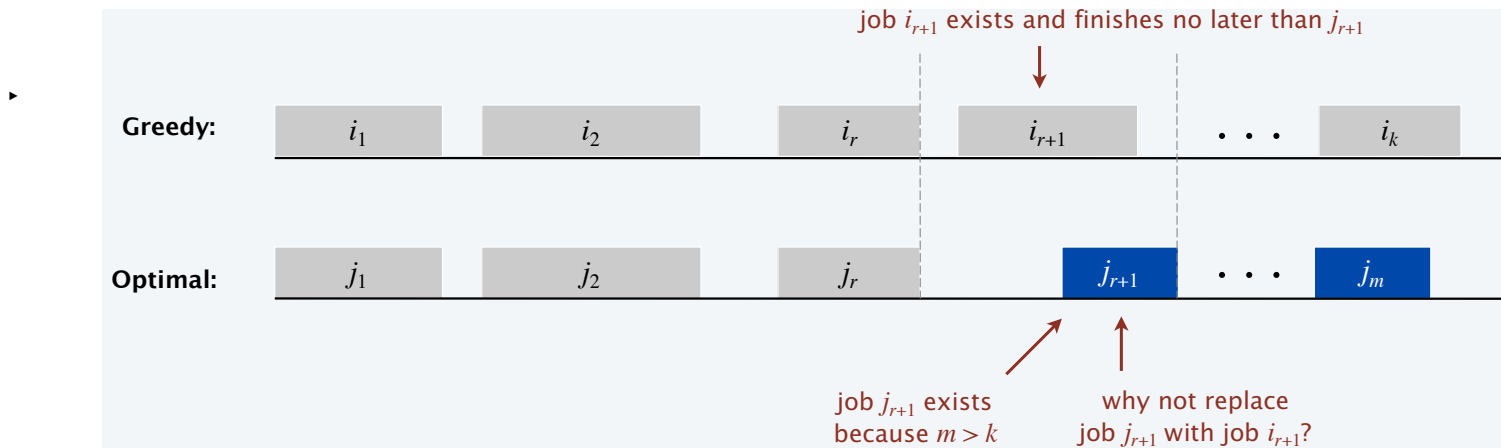
Interval Scheduling

- Quiz: Which is best? Sequence jobs by using an index, then add to schedule if compatible?
 - Earliest start time? (a,b,c,d,e,f,g,h)
 - Earliest finish time? (b,c,a,e,d,f,g,h)
 - Shortest interval? (c,b,e,h,f,g,d,a)
 - Least overlap? (h,b,c,g,a,e,f,d)



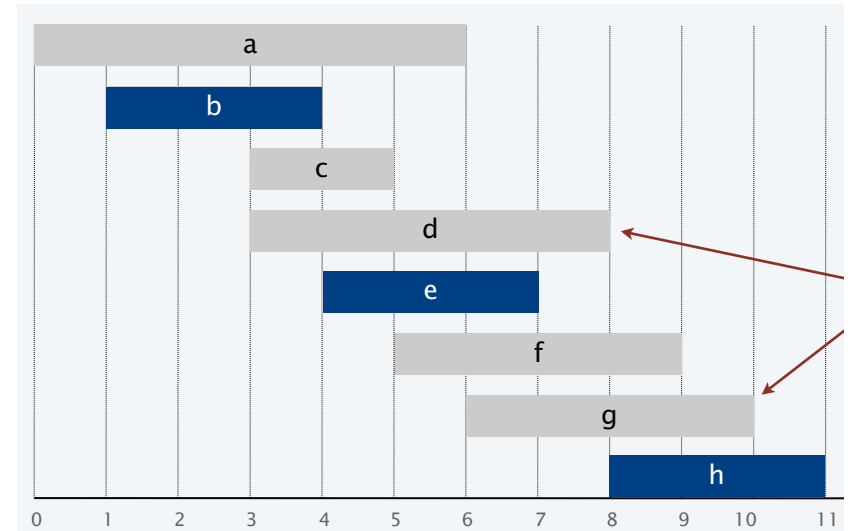
Interval Scheduling: Earliest finish time optimal

- Prove by contradiction
 - Let $i_1, i_2, \dots, i_k$ denote set of jobs selected by EFT, and let $j_1, j_2, \dots, j_m$ denote set of jobs in an optimal solution
 - Let r be largest value where $i_1 = j_1, \dots, i_r = j_r$ (schedules agree)
 - Note: i_{r+1} finishes no later than j_{r+1} (because of EFT choice)
 - So, can replace j_{r+1} with i_{r+1} and still be optimal for rest of schedule $J \rightarrow$ contradiction



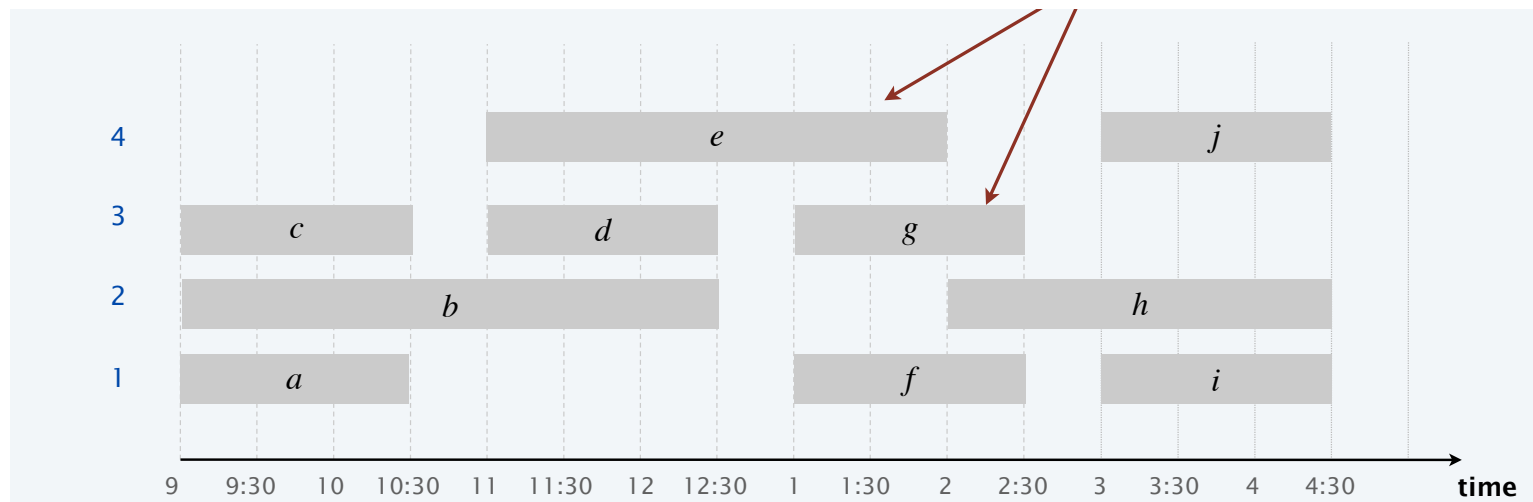
Parallel Job Scheduling

- Quiz: Which is best? Sequence jobs by using an index, then assign to first available processor?
 - Earliest start time? (a,b,c,d,e,f,g,h)
 - Earliest finish time? (b,c,a,e,d,f,g,h)
 - Shortest interval? (c,b,e,h,f,g,d,a)
 - Least overlap? (h,b,c,g,a,e,f,d)
 - ????



Scheduling Jobs on Parallel Processors

- Jobs $\{1, 2, \dots, N\}$ occur at fixed times, with start times $s(j)$, $f(j)$ for job j
- What is minimum number of parallel processors so we can schedule all jobs without conflict?
- The registrar problem: jobs are classes with start times, end times, and processors are classrooms (a bit more complicated if you add class sizes)



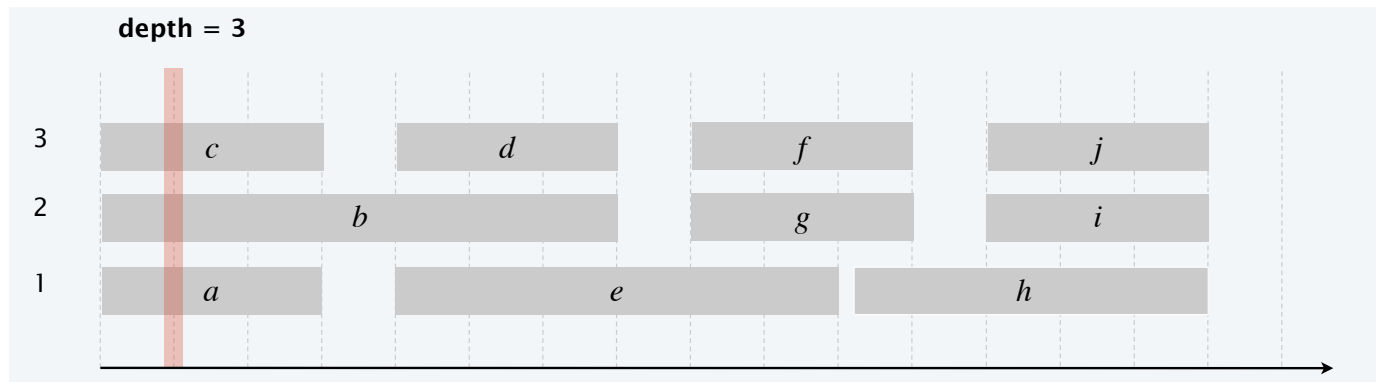
Optimal Algorithm

- Sequence by earliest start time: $s(1) \leq s(2) \leq \dots \leq s(n)$
 - Let d = number of allocated classrooms
 - For $j = 1$ to n :
 - If processor available at $s(j)$, assign $s(j)$ to processor
 - Else increase $d = d+1$, assign $s(j)$ to new processor
- Alternative, sequence by earliest finishing time...would also work
- $O(n \log(n))$ $\rightarrow$ need to sort

▸

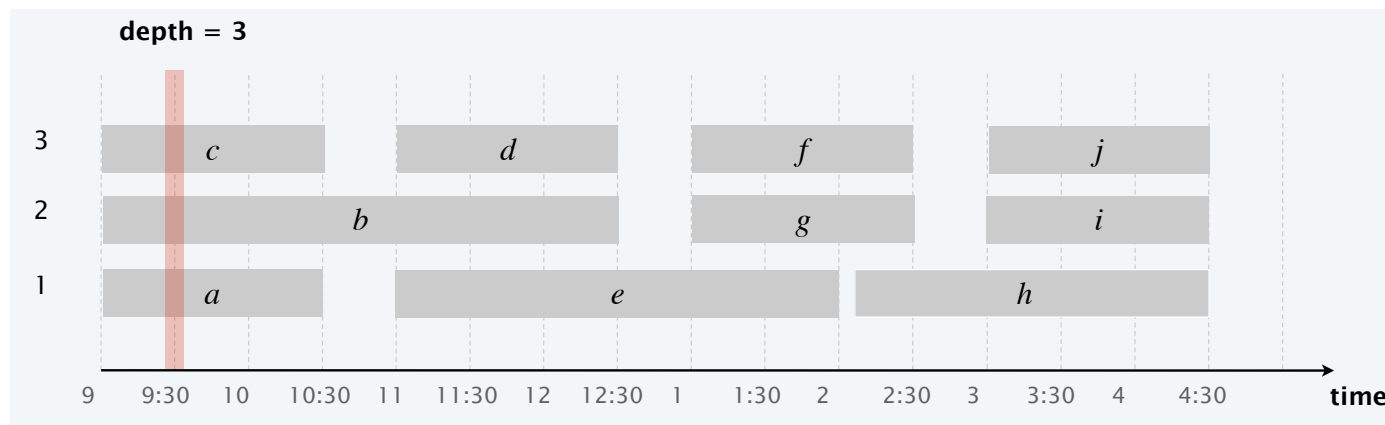
Concept: Depth of intervals

- Def. The depth of a set of open intervals is the maximum number of intervals that contain any given point.
- Key observation. Number of processors needed $\geq$ depth.
- Q. Does minimum number of processors needed always equal depth?
 - Yes! Moreover, earliest-start-time-first algorithm finds a schedule whose number of processors equals the depth



Optimality

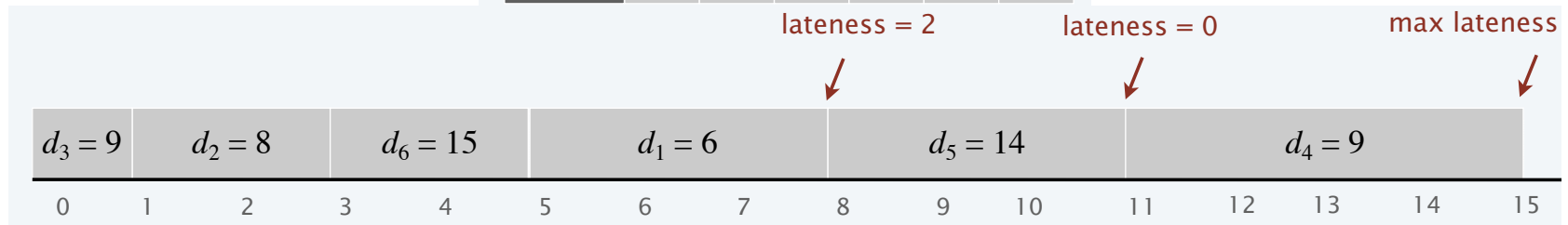
- If you must add a processor to increase d in earliest-start-time algorithm for job j at $s(j)$, then depth is at least $d+1$ at time $s(j) + \epsilon$
- Key observation. Number of processors needed $\geq$ depth
- Thus, EST algorithm achieves lower bound on number of processors, hence it is at least as good as any other algorithm $\rightarrow$ it is optimal



Scheduling to minimize lateness

- Jobs $\{1, 2, \dots, N\}$, with required processing times t_j and deadlines d_j
- If job j starts at time s , it finishes at $f_j = s + t_j$
- Lateness measure $\ell_j = \max\{0, f_j - d_j\}$
- Goal: schedule all jobs to minimize maximum lateness $L = \max_{j=1, \dots, N} \ell_j$

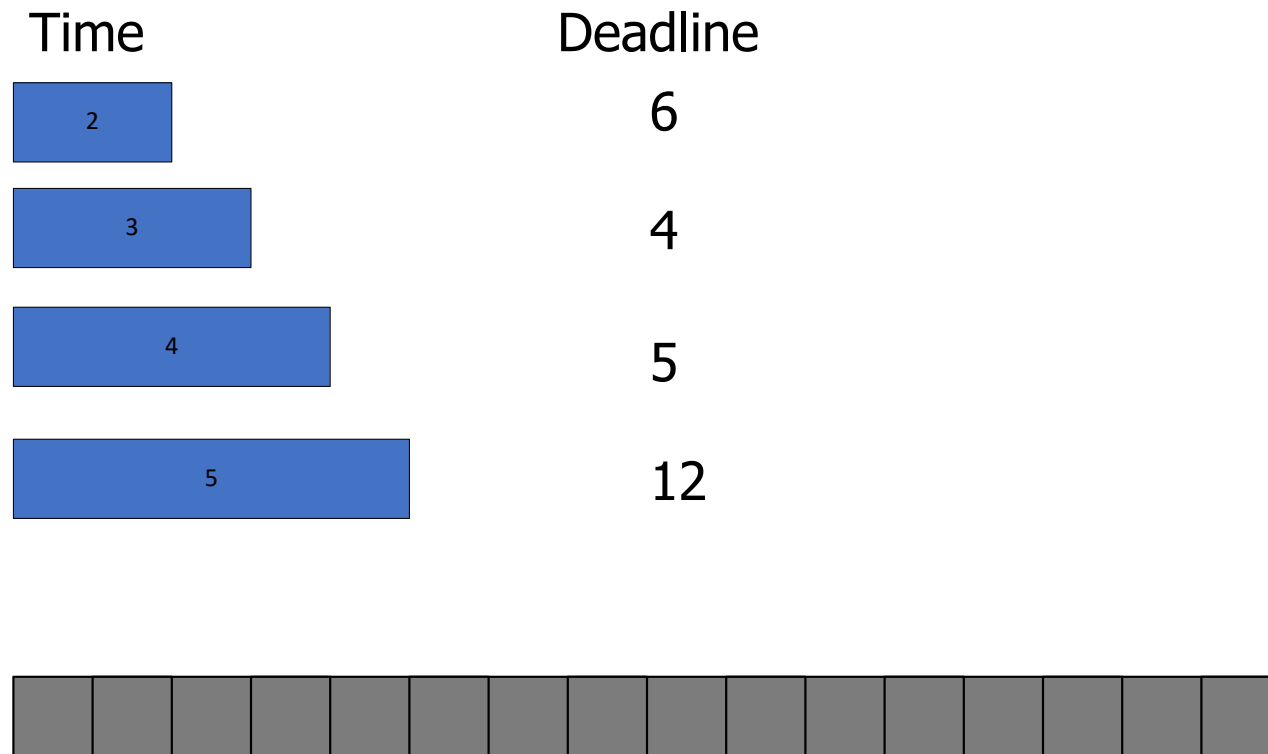
	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Example

Time	Deadline
	2
	4
	Lateness 1
	Lateness 3

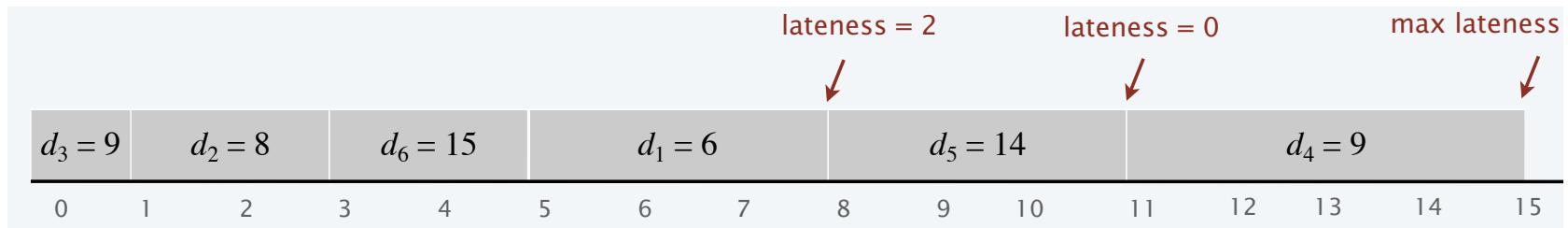
Determine the minimum lateness



Deadline Scheduling

- Quiz: Which is best? Sequence jobs by using an index?
 - Shortest processing time?
 - Earliest deadline first?
 - Increasing order of slack $d_j - t_j$?
 - Something else?

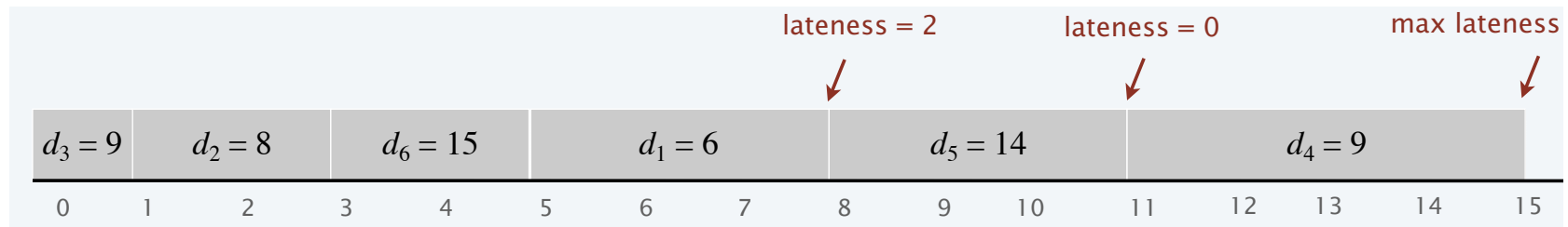
	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Greedy algorithm: Earliest deadline first

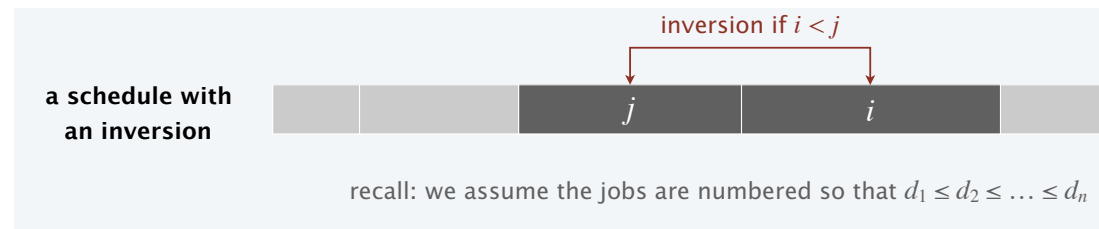
- Sort jobs by deadline, so $d_1 \leq d_2 \leq \dots \leq d_N$
- Start with $t = 0$
- For $j=1$ to N :
 - Schedule job j in $[t, t+d_j]$
 - Increase $t = t+d_j$

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



Observation: No idle time is optimal

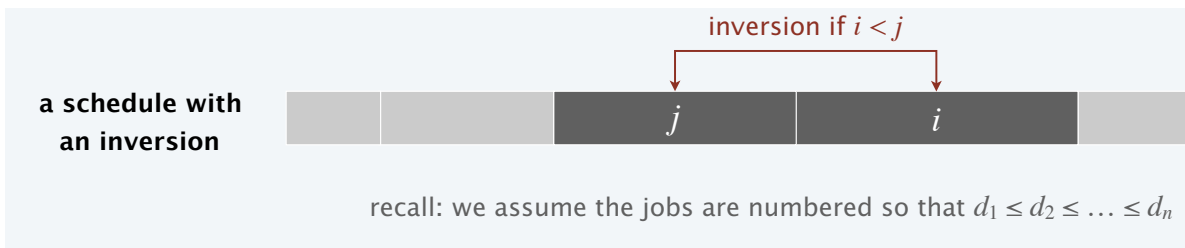
- There exist an optimal schedule with no idle time (by contradiction, easy to perturb to eliminate idle time)
 - Earliest deadline schedule has no idle time
- Definition: a schedule has an inversion if, for a particular pair $j < k$, job k is scheduled before job j
 - Note: if an inversion exists, then there must exist a pair of adjacent jobs in a schedule with inversion



- Earliest deadline schedule has no inversions

Minimizing Lateness

- Given a schedule S with inversions, exchanging the order of an adjacent, inverted pair j, k does not increase maximum lateness
- Proof: Let ℓ_m be the lateness of job m before the exchange, and ℓ'_m the lateness after the exchange
 - $\ell'_m = \ell_m$ for all $m \neq j, k$
 - $\ell'_j \leq \ell_j$ because we have moved it up
 - If job k is now late, $\ell'_k = f'_k - d_k = f_j - d_k \leq f_j - d_j = \ell_j$ (earliest deadline)
 - So, max lateness is not increased



Optimality

- Earliest deadline first is optimal
- Proof:
 - Assume there is an optimal schedule that is not earliest deadline first that has the fewest number of inversions among optimal schedules
 - If number of inversions is 0, it is the EDF solution.
 - If not, there must be an adjacent inversion
 - Swapping the inversion does not increase the maximum lateness, and reduces the number of inversions, so we generate an optimal schedule which has less inversions
 - Contradiction

Example

- Jobs: 1, 2, 3, 4, 5;
- Processing times: 5, 4, 3, 2, 3
- Algorithm: Sort jobs by processing time: 4, 3, 5, 2, 1, schedule in order

4	4	3	3	3	5	5	5	2	2	2	2	1	1	1	1	1	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

- Finishing times: $2 + 5 + 8 + 12 + 17 = 44$
- Alternative schedule: schedule by number: 1, 2, 3, 4, 5

1	1	1	1	1	2	2	2	2	3	3	3	4	4	5	5	5	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

- Finishing times: $5 + 9 + 12 + 14 + 17 = 57$

Scheduling with Deadlines

- Single server, N jobs with the same processing time
 - Job j has **unit processing time**, hard deadline $d(j)$, value $V(j)$ if job completed on time
 - Objective: Find sequence of jobs that will be scheduled to maximize value completed on time
 - Concept: a set of jobs $\{j_1, \dots, j_k\}$ is feasible $\iff$ there is an ordering of these jobs where each job is completed before its deadline
 - If a set of jobs is feasible, scheduling them by earliest deadline first is sufficient $\rightarrow$ simple to show
 - Can also schedule them just in time — schedule job j as close as possible to its deadline $d(j)$

Determining Best Feasible Set

- Algorithm:
 - Initialize feasible set S as empty; sort the jobs by decreasing value $V(j)$, into order $j_1, \dots, j_N$
 - Initialize step $k = 1$
 - At step k , add job j_k to S , and see if S is still a feasible set
 - If S is not feasible, remove j_k from S
 - If $k < N$, increment $k = k+1$ and repeat
- Claim: this will be an optimal feasible set
 - Important feature: every job takes the same amount of time

String Matching and Tries

- Consider the following problem:
 - Given a string T and k nonempty strings $P_1, P_2, \dots, P_k$, find all occurrences of $P_1, P_2, \dots, P_k$ in T .
 - T is called the text string and $P_1, P_2, \dots, P_k$ are called pattern strings.
 - This problem was originally studied in the context of compiling indexes, but has found applications in computer security and computational genomics
- Simpler problem: $k = 1$
 - Knuth-Morris-Pratt (KMP) algorithm (CLRS)
- Harder problem: $k > 1$
 - Aho-Corasick tries

Single Pattern Matching

- Given a string $T[0:n-1]$ of n characters in finite alphabet set S
- Nonempty string $P = P[0:m-1]$ of characters in S
- Pattern P occurs with shift s in $T[s:s+m-1] = P[0:m-1]$
- Problem: Find all shifts s where Pattern occurs with shift s
- Naive algorithm:
 - For s in 0 to $n-m$:
 - If $T[s:s+m-1] = P[0:m-1]$, add s to set of shifts where pattern occurs
 - Complexity: $O(m(n-m))$

Knuth-Morris-Pratt Algorithm

- Idea: exploit that whenever we find a mismatch, we have already looked at a subset of the pattern
 - $T[] = \text{"AAAA A AAAAAAAAAAAAB"}$ ($n=20$)
 - $P[] = \text{"AAAA B"}$ ($m=6$)
 - If we get a mismatch on $T[5]$, we don't have to start searching from $s=1$ for the next match! We know no shift can match before $s = 6$
- Idea: patterns have a prefix function
 - A string w is a **proper prefix** of a string x if $x = w + v$ for some string non-empty string v , where $+$ is concatenation
 - A string w is a **proper suffix** of a string x if $x = v + w$ for some non-empty string v

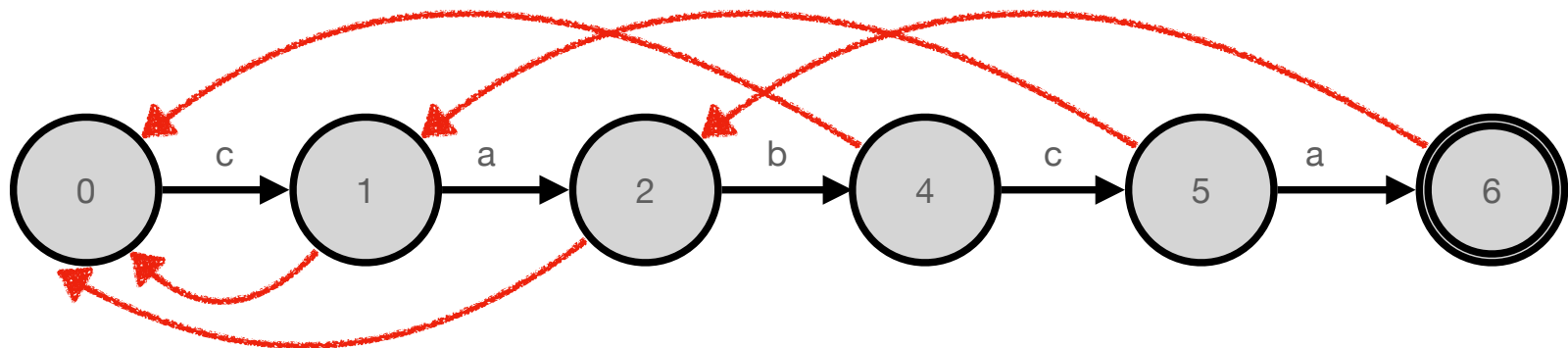
Knuth-Morris-Pratt Algorithm

- Algorithm: preprocess pattern $P[0:m-1]$ to compute prefix function $\pi[j]$
 - $\pi[j]$ = longest proper prefix of $P[0:j]$ that is also a suffix of $P[0:j]$, for $j = 0$ to $m-1$
 - Examples: $P = \text{"AAAA"} \rightarrow \pi[] = [0, 1, 2, 3]$
 - $P = \text{"ABCDE"} \rightarrow \pi[] = [0, 0, 0, 0, 0]$
 - $P = \text{"AABAACAABAA"} \rightarrow \pi[] = [0, 1, 0, 1, 2, 0, 1, 2, 3, 4, 5]$
 - $P = \text{"AAACAAAAAC"} \rightarrow \pi[] = ?$
 - $P = \text{"AAABAAA"} \rightarrow \pi[] = ?$
- We can use the prefix function to figure out how much to shift the pattern forward for matching!

A Different View of the Prefix Function

Finite State Automata

Pattern: c a b c a $\pi() = [0,0,0,1,2]$



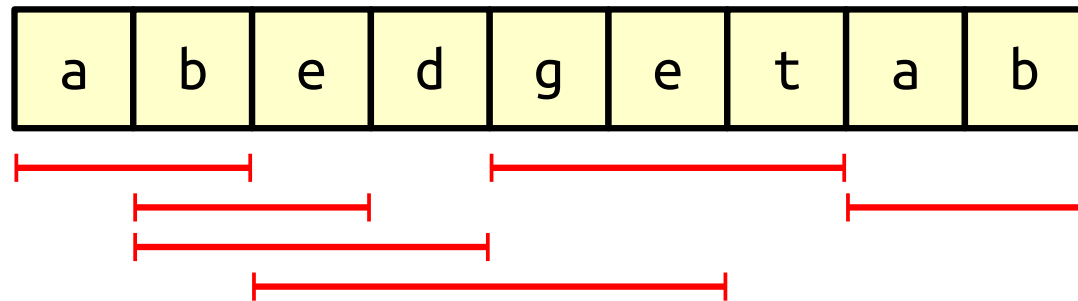
- If letter match, advance to next state, check next character.
- Else, follow red link to state, check current character.
- If reach double circle, output match, follow red link to previous state and continue searching current character.

Search for Multiple Strings

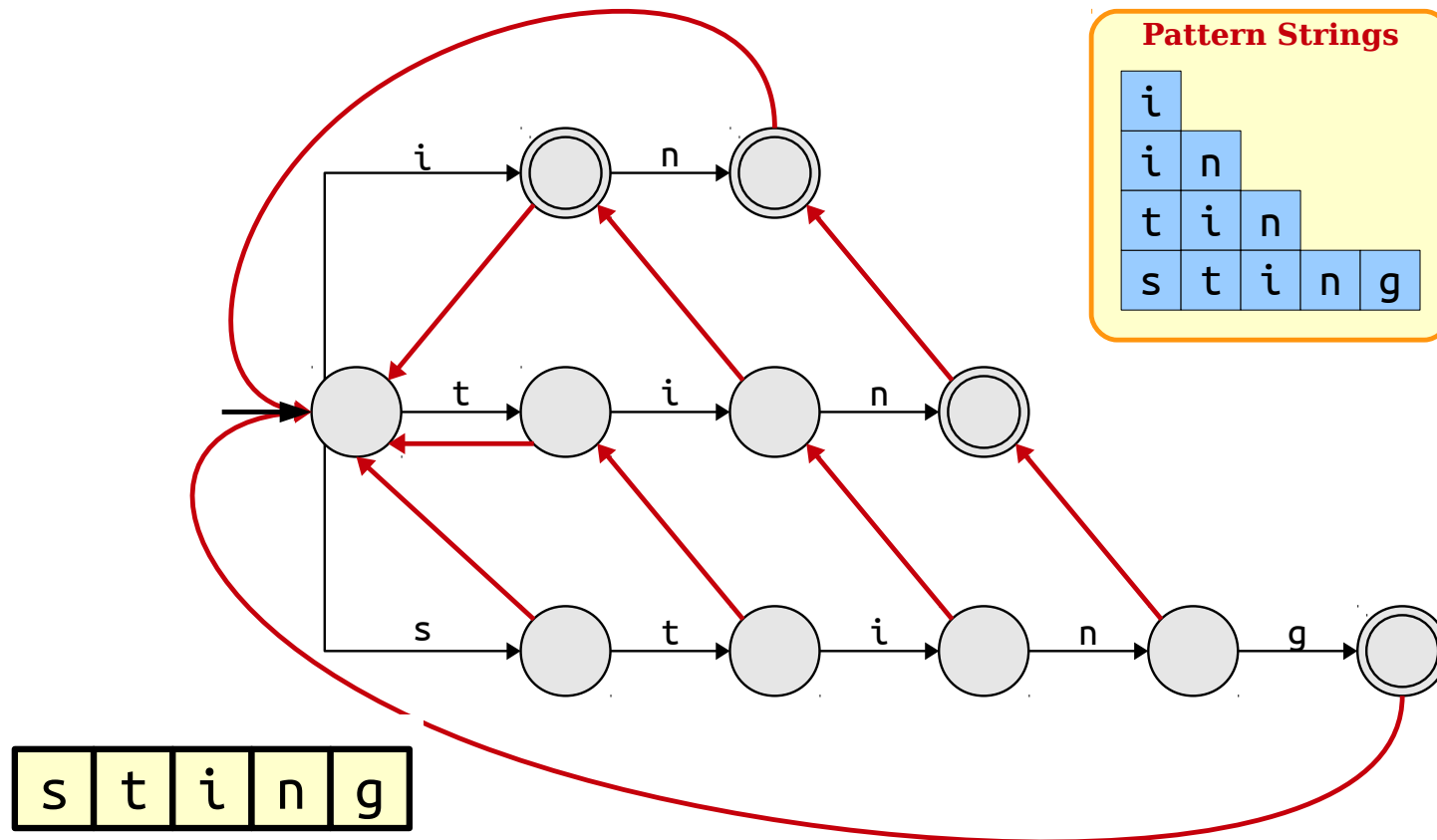
- KMP good for 1 pattern P
- What if we are looking for k patterns $P_1, P_2, \dots, P_k$?

Pattern Strings

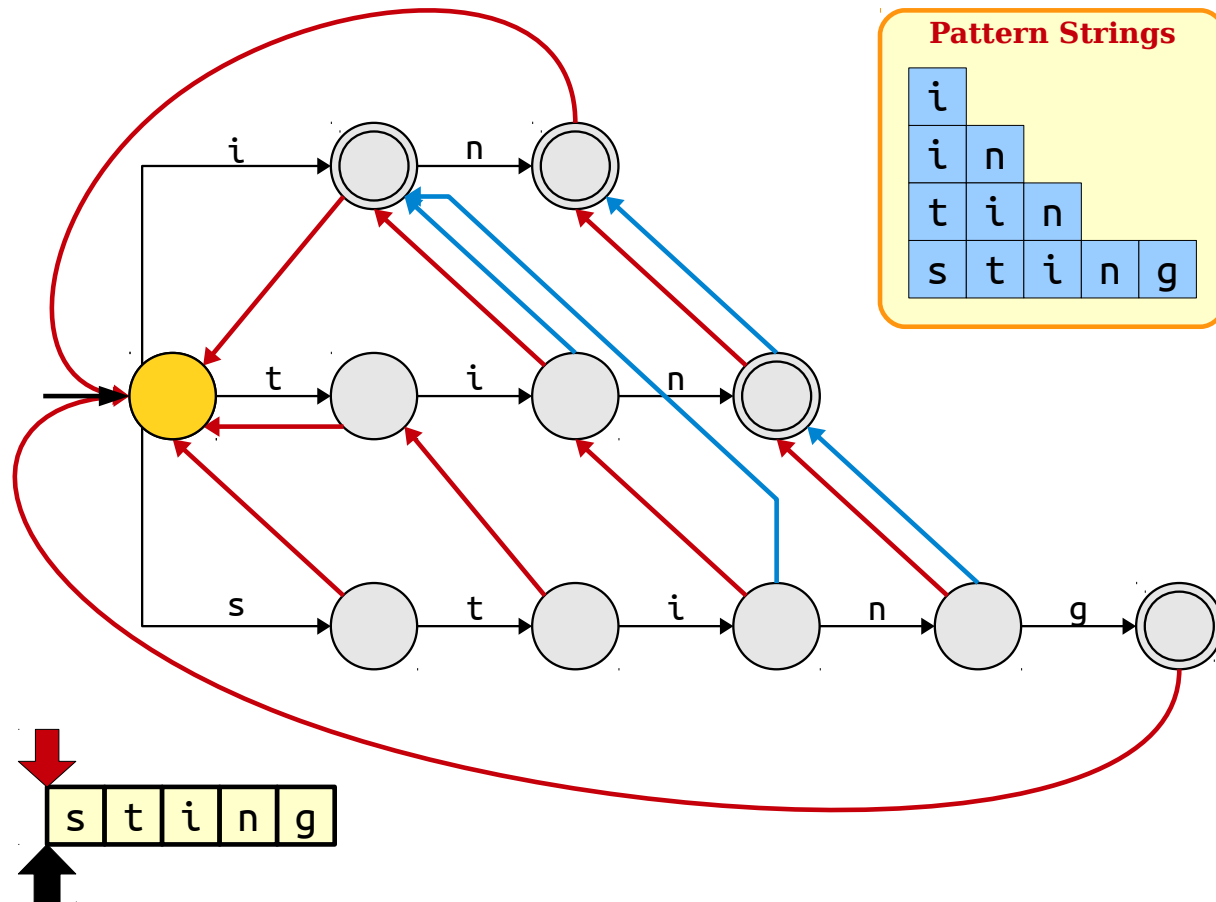
a	b			
a	b	o	u	t
a	t			
a	t	e		
b	e			
b	e	d		
e	d	g	e	
g	e	t		



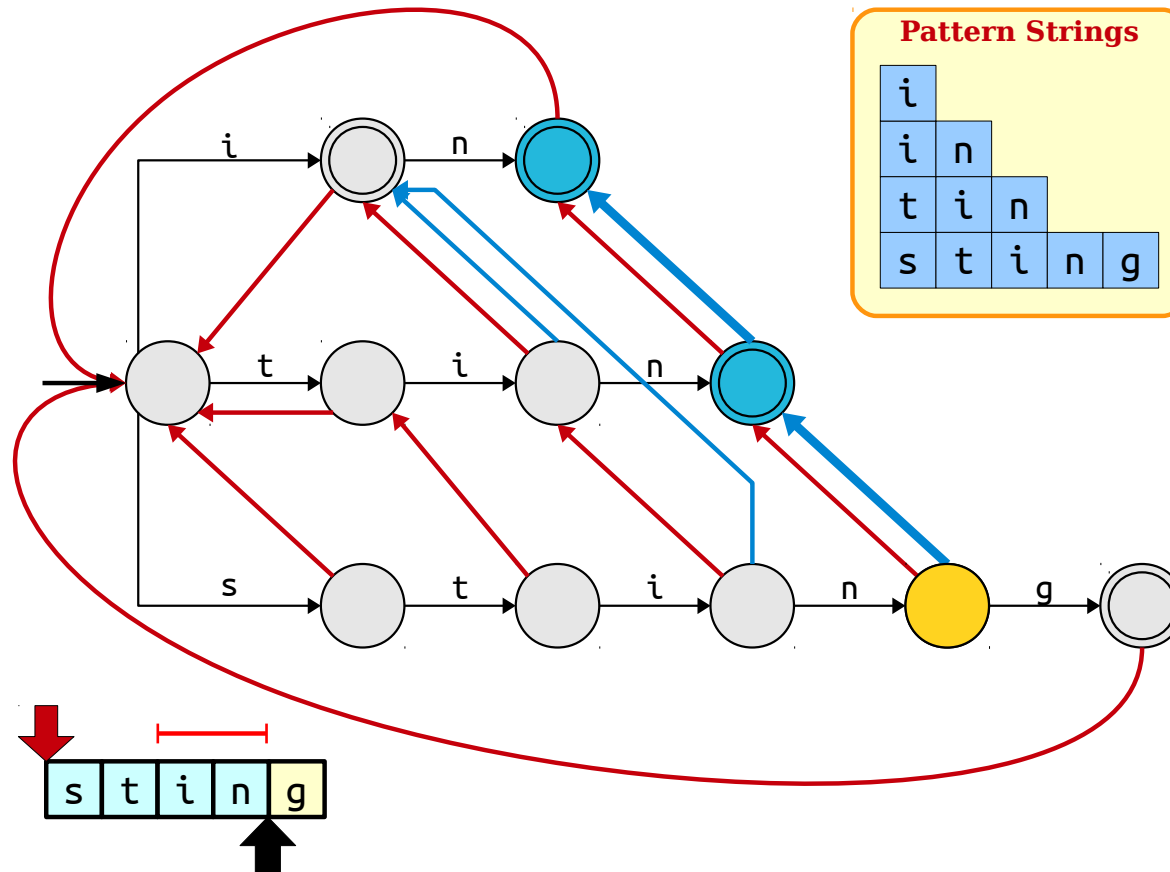
A Possible Problem: Detecting words inside other words



Output Links



Output Links



Dynamic Programming

- Pioneered by Richard Bellman (late 1950s)
 - Extended significantly and is still seeing tremendous development
 - Most recent popular press applications: AlphaZero, AlphaGo, AlphaStar
- A general approach for breaking solutions of large problems into sequence of solutions of smaller problems
 - Originally developed for making decisions over time by decomposing the problem into making such decisions sequentially time by time
 - Dynamic programming: “planning over time”
- We have used dynamic programming already
 - Bellman-Ford, Floyd-Warshall
 - Want to study other applications of dynamic programming to understand technique

Knapsack Problem

- Given n items
 - Item j has value $V(j) > 0$, size $c(j) > 0$ (assume integer $c(j)$)
- Given a box of size $C > 0$ (integer-valued)
- Select items that fit together in the box, and maximize the total value

$$\max_{x_i \in \{0,1\}} \sum_{i=1}^n V(i) x_i$$

$$\text{Subject to the constraint } \sum_{i=1}^n c(i) x_i \leq C$$

x_i are indicator variables: 1 means item goes in the box, 0 item stays out

Select items that fit together in the box, and maximize the total value

Fractional Knapsack Problem

- Given n items
 - Item j has value $V(j) > 0$, size $c(j) > 0$
- Given a box of size C
- Select fractions items that fit together in the box, and maximize the total value

$$\max_{x_i \in [0,1]} \sum_{i=1}^n V(i) x_i$$

$$\text{Subject to the constraint } \sum_{i=1}^n c(i) x_i \leq C$$

x_i are fraction of item i that goes in the box

Fractional Knapsack Problem

- Greedy solution

Rank items by diminishing value per unit size: $\frac{V(j)}{c(j)}$

- Insert items in order; assume j is the last full item that fits in the box

$$x_1, x_2, \dots, x_j = 1; \quad x_{j+1} = \left(C - \sum_{i=1}^j c(i)\right) / c(j+1)$$

- Easy proof by contradiction; any solution that does not satisfy this can be replaced by a solution that satisfies this with at least as much value
- Requires “partial credit”

Integer Knapsack : Dynamic Programming

- Value function: $Opt(j,k)$ is best value considering items 1, ..., j only, for capacity $C = k$

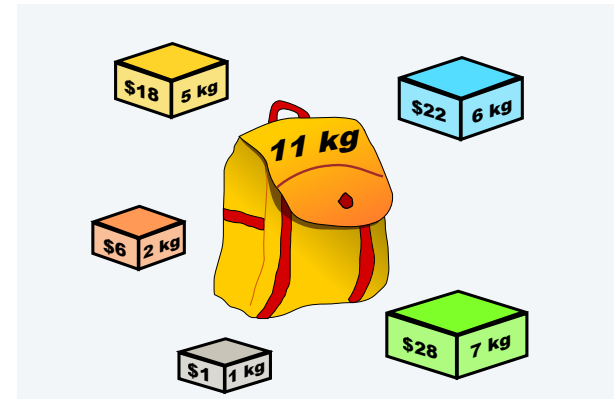
Easy initialization: $Opt(1,k) = \begin{cases} 0, & k < c(1); \\ V(1), & k \geq c(1) \end{cases}$

- If we consider an additional item $j+1$, for a capacity $C = k$, if we fit that item, then other items have to fit in remaining capacity $C = k - c(j+1)$
- Recursion

$$Opt(j+1,k) = \begin{cases} Opt(j,k), & k < c(j+1) \\ \max\{Opt(j,k), V(j+1) + Opt(j, k - c(j+1))\} & \text{otherwise} \end{cases}$$

Example

Item	V(j)	w(j)	V(j)/w(j)
1	1	1	1
2	6	2	3
3	18	5	3.6
4	22	6	3.67
5	28	7	4



j\k	0	1	2	3	4	5	6	7	8	9	10	11
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	25	25	25	25
4	0	1	6	7	7	7	22	24	28	29	29	40
5	0	1	6	7	7	7	22	28	29	34	35	40

Integer Knapsack Complexity

- Complexity: $\Theta(nC)$
 - Number of rows: n
 - Number of columns: C
 - Computation per entry: $O(1)$
- Complexity is **not polynomial** (depends on C , so pseudo-polynomial)
- Space required is also $\Theta(nC)$
- Note algorithm depends critically on the fact that sizes $c(j)$ are integers
 - Can handle non-integer $c(j)$ with a lot more notation

Sequence Alignment

- How similar are two sequences of symbols?
 - Example: occurrence and occurrence

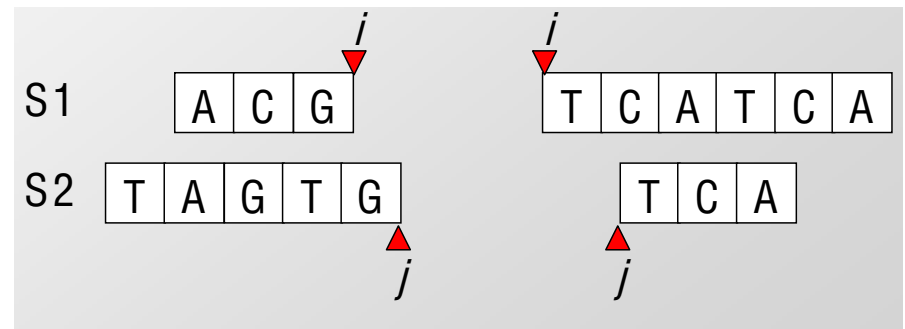
o	c	u	r	r	a	n	c	e	-
o	c	c	u	r	r	e	n	c	e
6 mismatches, 1 gap									

o	c	-	u	r	r	a	n	c	e
o	c	c	u	r	r	e	n	c	e
1 mismatch, 1 gap									

- Applications: Bioinformatics, spell correction, machine translation, speech recognition, information extraction

Edit Distance

- Concept due to Levenshtein 1966, Needleman–Wunsch 1970
- Scoring function
 - Cost of mutation (mismatch)
 - $s(x,y)$ is cost of matching $x \neq y$
 - Cost of insertion/deletion
 - δ is cost of matching x to a gap, or matching y to a gap
 - Reward of correct match
 - $s(x, y)$ is value of correctly matching when $x = y$
- Complex search problem
 - For sequences of length 100, number of possible matches is $9 \cdot 10^{58}$



Sequence Alignment Optimization

- Cost of M is

$$Cost(M) = \sum_{i,j \text{ matched}} s(x_i, y_j) + \sum_{i: x_i \text{ unmatched}} \delta + \sum_{j: y_j \text{ unmatched}} \delta$$

- Useful structure: score is additive $\rightarrow$
similar structure to path length as additive over edge lengths
 - For a given split (i,j), we have score of best alignment $x[1:n]$ and $y[1:n]$ is sum of scores of best alignment $x[1:i]$, $y[1:j]$ + best alignment $x[i+1:m]$, $y[j+1:n]$
 - This will allow us to use dynamic programming

x_1	x_2	x_3	x_4	x_5	x_6	
C	T	A	C	C	—	G
—	T	A	C	A	T	G
	y_1	y_2	y_3	y_4	y_5	y_6

an alignment of CTACCG and TACATG

$M = \{ x_2-y_1, x_3-y_2, x_4-y_3, x_5-y_4, x_6-y_6 \}$

Spokesperson confirms	senior government	adviser was found
Spokesperson said	the senior	adviser was found

Dynamic Programming (Needleman-Wunsch)

- Let $OPT(i, j)$ = minimum cost of aligning prefix strings $x_1x_2\cdots x_i, y_1y_2\cdots y_j$
- Goal. Is to compute $OPT(m,n)$
- Idea: Assume we know $OPT(i-1, j-1)$, $OPT(i,j-1)$, and $OPT(i-1,j)$:
 - Case 1. $OPT(i, j)$ matches $x_i \rightarrow y_j$: $Opt(i, j) = s(x_i, y_j) + OPT(i-1, j-1)$
 - Case 2a. $OPT(i, j)$ leaves x_i unmatched: $Opt(i, j) = \delta + OPT(i-1, j)$
 - Case 2b. $OPT(i, j)$ leaves y_j unmatched: $Opt(i, j) = \delta + OPT(i, j-1)$
- Initially, $Opt(i,0) = i\delta$; $Opt(0,j) = j\delta$
- Iteration:
$$Opt(i, j) = \min \{s(x_i, y_j) + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1)\}$$
$$Ptr(i, j) = \{\text{diag, up, left}\} \text{ corresponding to which term is minimized}$$

Example

\

Table 1

	j	0	1	2	3	4	5		skip	1
i			c	a	d	b	d		mismatch	1
0		0	1	2	3	4	5		Correct match	-2
1	a	1	1	-1	0	1	2			
2	c	2	-1	0	0	1	2		s = acbcd b	
3	b	3	0	0	1	-2	-1		t = ca dbd	
4	c	4	1	1	1	-1	-1			
5	d	5	2	2	-1	0	-3			
6	b	6	3	3	0	-3	-2			

Optimal Match: Backtrack Pointers

Table 1

	j	0	1	2	3	4	5
i			c	a	d	b	d
0		0	1	2	3	4	5
1	a	1	1	-1	0	1	2
2	c	2	-1	0	0	1	2
3	b	3	0	0	1	-2	-1
4	c	4	1	1	1	-1	-1
5	d	5	2	2	-1	0	-3
6	b	6	3	3	0	-3	-2

The table shows the scores for aligning the sequence 'cabd' (rows 1-5) with the sequence 'cabd' (columns 1-5). The optimal path is highlighted with blue circles and red arrows, starting from the bottom-right cell (6, 5) and moving back to the top-left cell (0, 0).

Optimal Path (Backtrack Pointers):

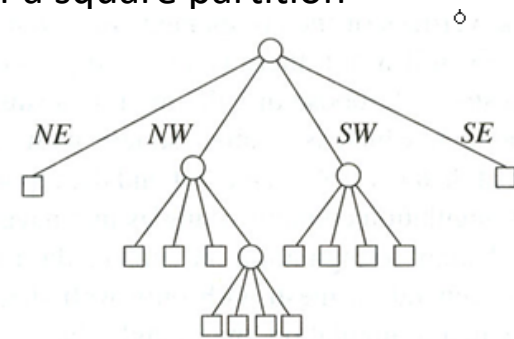
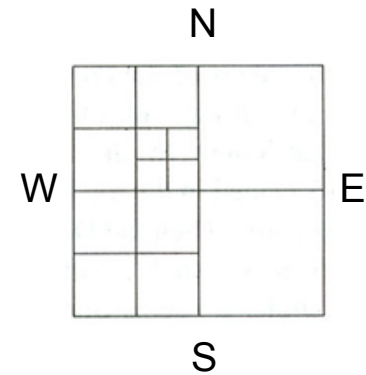
- Cell (6, 5) points to (5, 4)
- Cell (5, 4) points to (4, 3)
- Cell (4, 3) points to (3, 2)
- Cell (3, 2) points to (2, 1)
- Cell (2, 1) points to (1, 0)
- Cell (1, 0) points to (0, 0)

Sequence Matching Complexity

- Need to complete table of m by n
 - Length of x : m , length of y : n
- Computational complexity $O(mn)$
 - $O(1)$ operations to compute new element
 - Polynomial!
- Still, may be too slow for long DNA sequences
 - 50,000 genes...
- Search for faster approximate algorithms is topic of current research

Point-Region Quadtree

- PR Quadtrees are tries
 - Trie: Decomposition based on equal division of the key space
 - Shaped like a tree, with each internal node with 4 children (some empty)
- Every internal node corresponds to a region, with midpoint used for navigation
- Leaves correspond to 2-D points
- The children of a node correspond to the four quadrants of a square partition of a region
 - The children of a node are labelled NE, NW, SW, and SE to indicate to which quadrant they correspond
- If a leaf contains more than one point, it splits into 4 subregions
- Need rule to break ties: arbitrary prefer N to S, E to W
- 3-D variant: Octrees

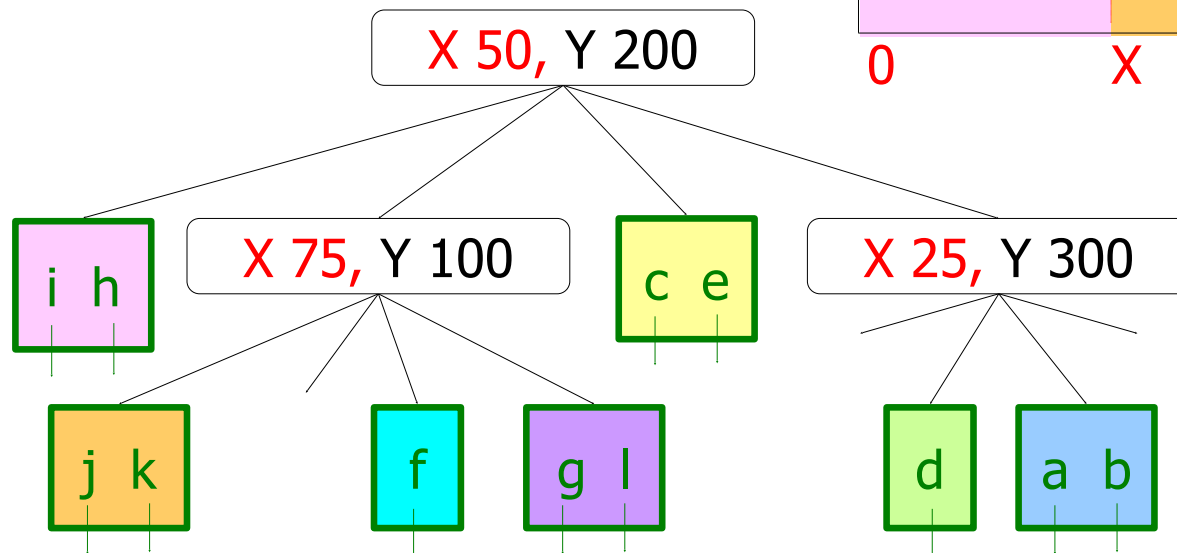
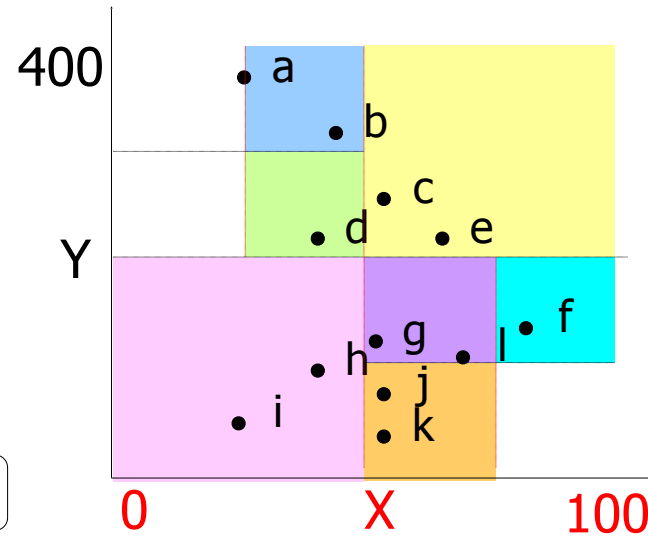


Quadtree Construction

Input: point set P

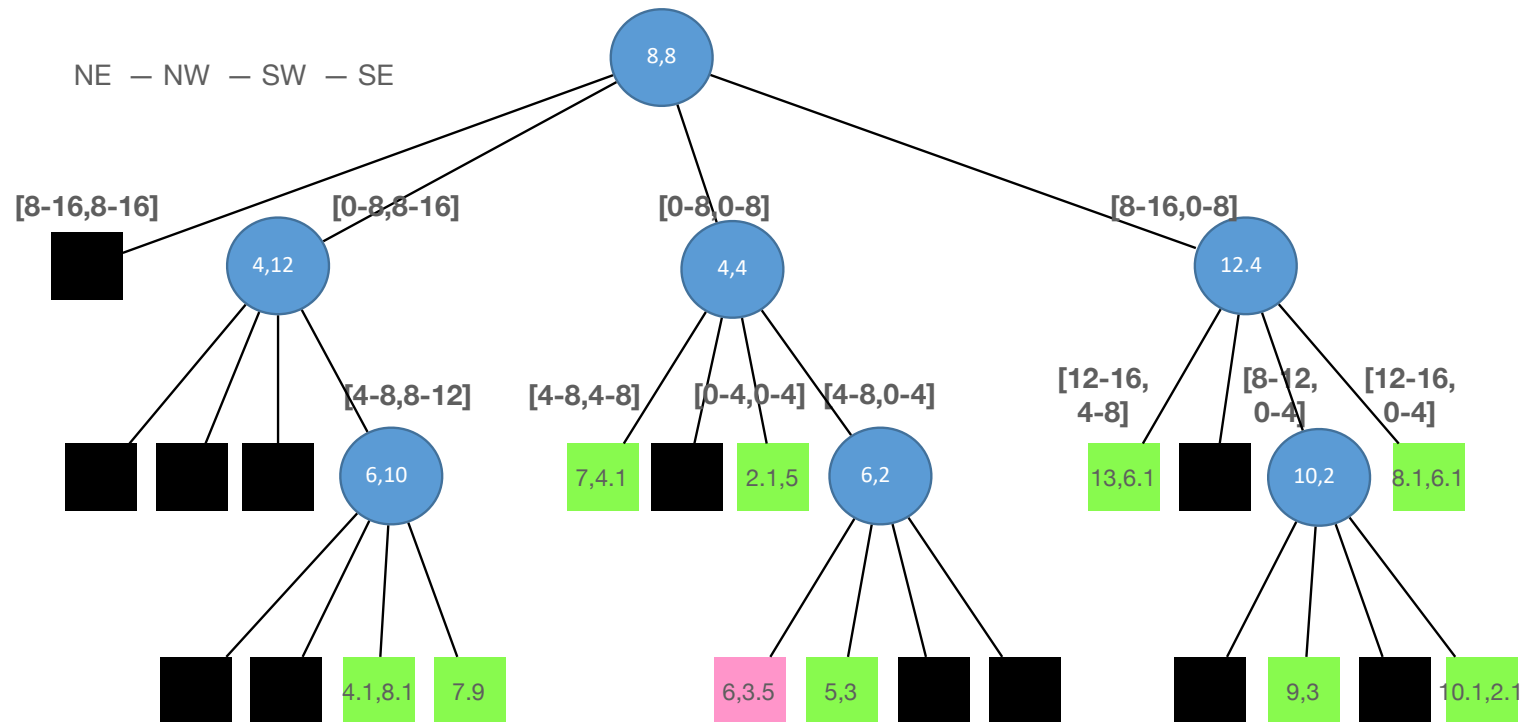
while Some cell C contains more than 1 point **do**
 Split cell C

end



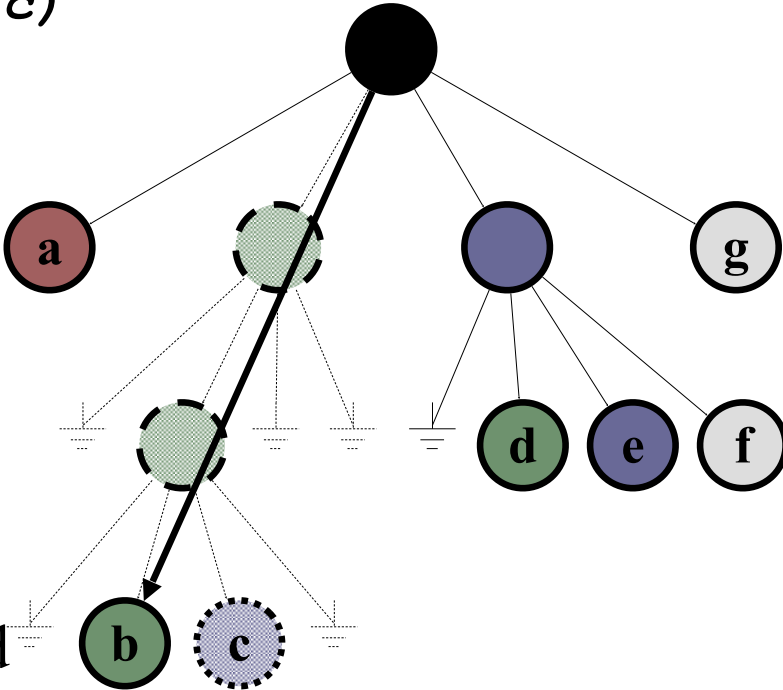
Insert Example - 2

- Insert (6,3.5)



/

/



- Find and delete the node.
- If its parent has just one child remaining, collapse leaf to parent
- Propagate upward

k-d trees (k-dimensional trees)

Main ideas:

- Generalize Binary Search Trees to k-dimensional data
- k-d tree: binary search tree where search decisions are made based on different coordinates at each level
 - Root is level 0
 - At level i , splitting decision is made based on coordinate $(i \bmod k) + 1$
- Property: node at i level use discriminator index $j = (i \bmod k) + 1$ with key value x_j
 - Right descendants $(x'_1, \dots, x'_k)$ must have $x'_j \geq x_j$
 - Left descendants $(x''_1, \dots, x''_k)$ must have $x''_j < x_j$

2-dimensional kd-trees

A data structure to support nearest neighbor and range queries in $\mathbb{R}^2$.

- Not the most efficient solution in theory.
- Everyone uses it in practice.

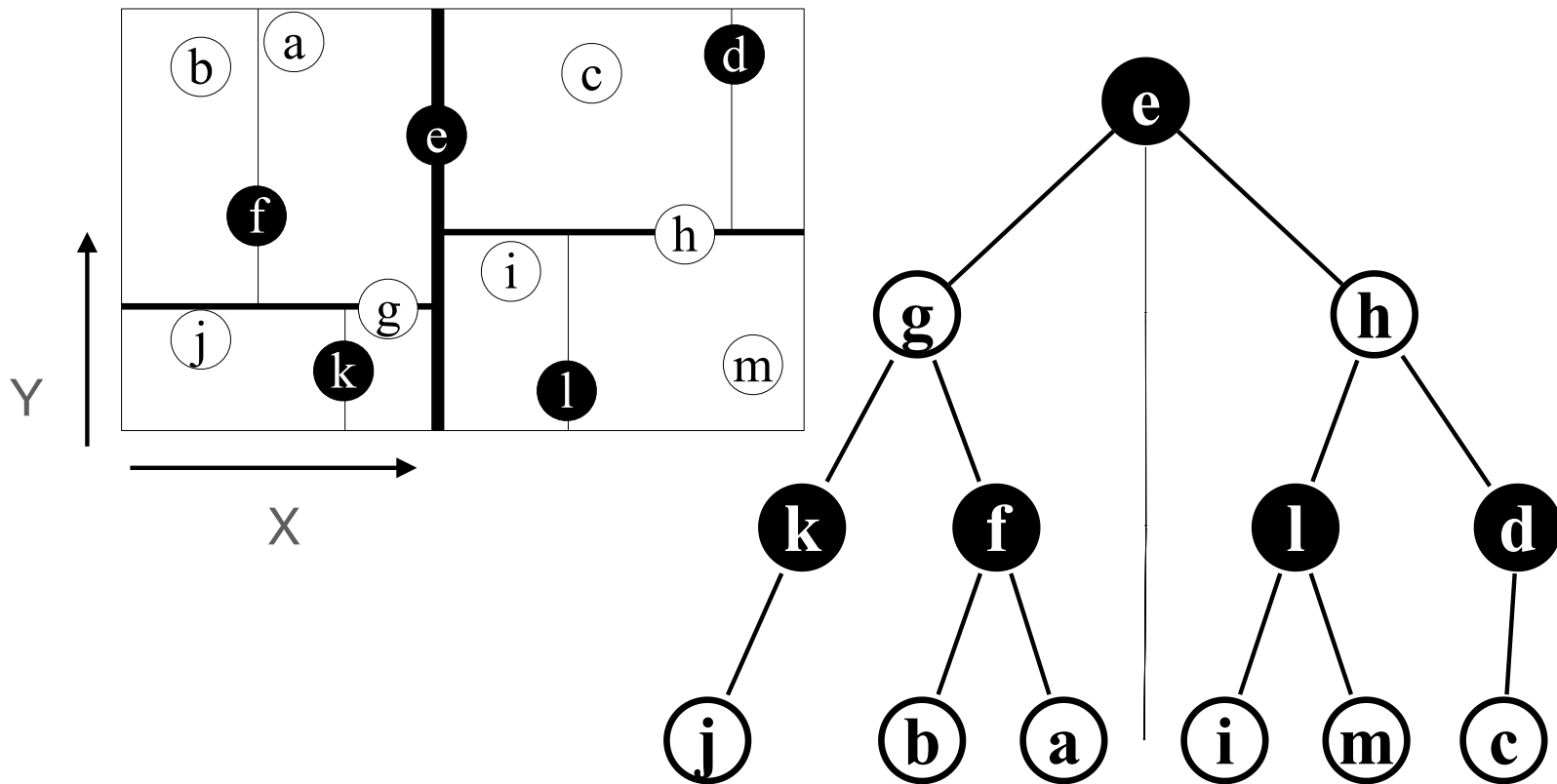
Algorithm: Batch construction

- Choose x or y coordinate (alternate).
- Choose the median of the coordinate; this defines a horizontal or vertical line.
- Recurse on both sides until there is only one point left, which is stored as a leaf.

We get a binary tree

- Size $O(n)$
- Construction time $O(n \log n)$
- Depth $O(\log n)$
- K-NN query time: $O(n^{1/2} + k)$...under many assumptions

k -D Tree



Kd Trees Can Be Inefficient if built sequentially (but not when built in batch!)

insert(<5,0>)

insert(<6,9>)

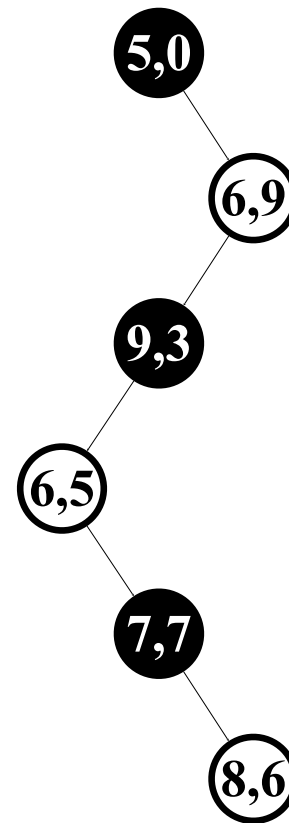
insert(<9,3>)

insert(<6,5>)

insert(<7,7>)

insert(<8,6>)

Incremental inserts not good...



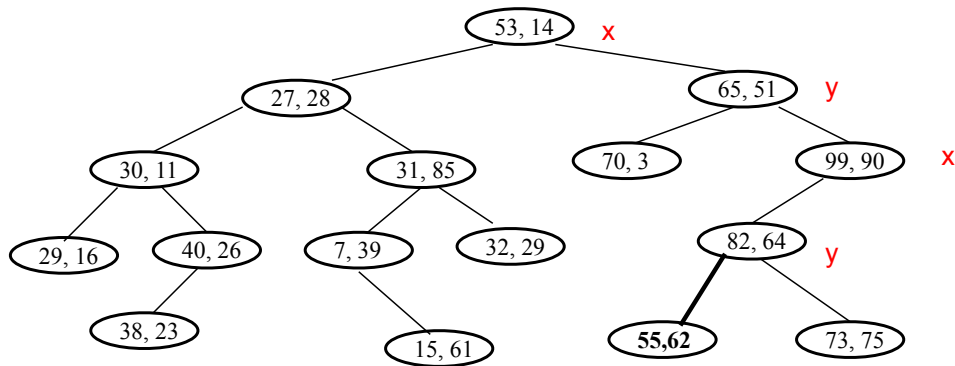
Delete data

- Suppose we need to remove $\mathbf{p} = (a, b)$
 - Find node $\mathbf{t}$ which contains $\mathbf{p}$
 - If $\mathbf{t}$ is a leaf node, replace it by null
 - Otherwise, find a replacement node $\mathbf{r} = (c, d)$ – see below!
 - Replace (a, b) by (c, d)
 - Remove $\mathbf{r}$
- Finding the replacement $\mathbf{r} = (c, d)$
 - If $\mathbf{t}$ has a right child, use the successor*
 - Otherwise, use node with minimum value* in the left subtree
 - Move right child of that node as appropriate

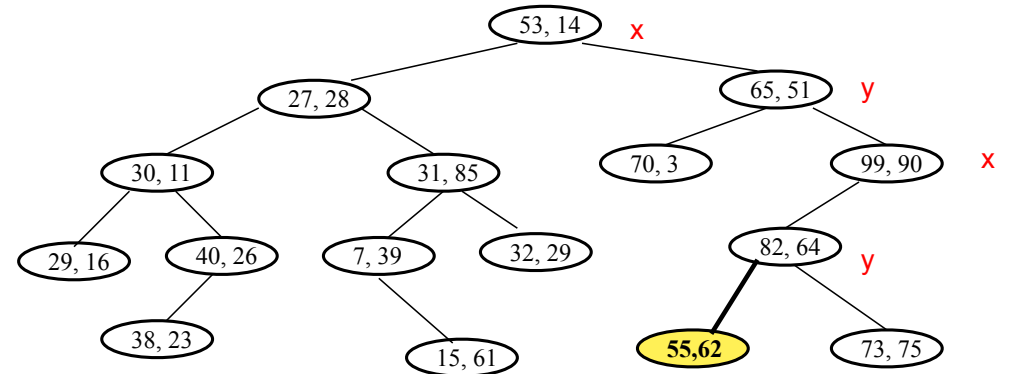
*(depending on what axis the node discriminates)

Delete data (cont'd)

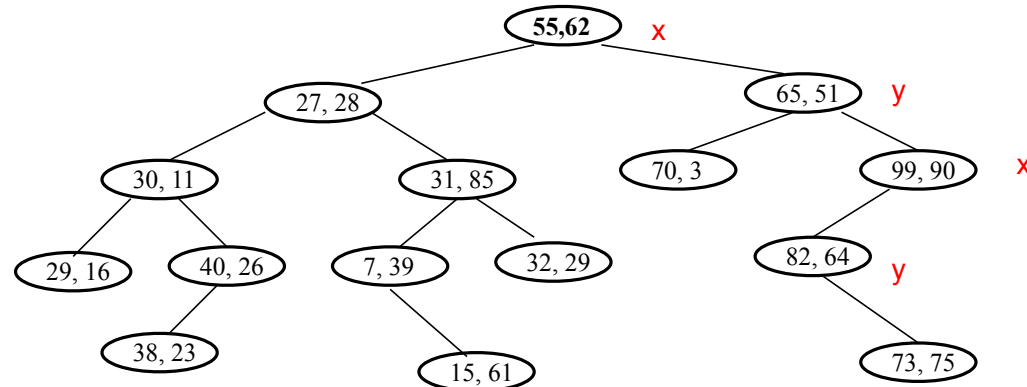
Delete (53,14)



Find min x on right



Swap it



Quad Trees vs. k -D Trees

- k -D Trees
 - Density balanced trees
 - Number of nodes is $O(n)$ where n is the number of points
 - Height of the tree is $O(\log n)$ *with batch insertion*
 - Supports insert, delete, find, nearest neighbor, range queries, but delete is hard
- Quad Trees
 - Number of nodes is $O(n(1 + \log(\Delta/n)))$ where n is the number of points and Δ is the ratio of the width (or height) of the key space and the smallest distance between two points
 - Height of the tree is $O(\log n + \log \Delta)$
 - Supports insert, delete, find, nearest neighbor, range queries

R-tree

- In multidimensional space, there is no unique ordering! Not possible to use B+-trees☹
- [Guttman 84] R-tree!
- Group objects close in space in the same node
 - => guaranteed page utilization
 - => easy insertion/split algorithms.
 - (only deal with Minimum Bounding Rectangles - **MBRs**)

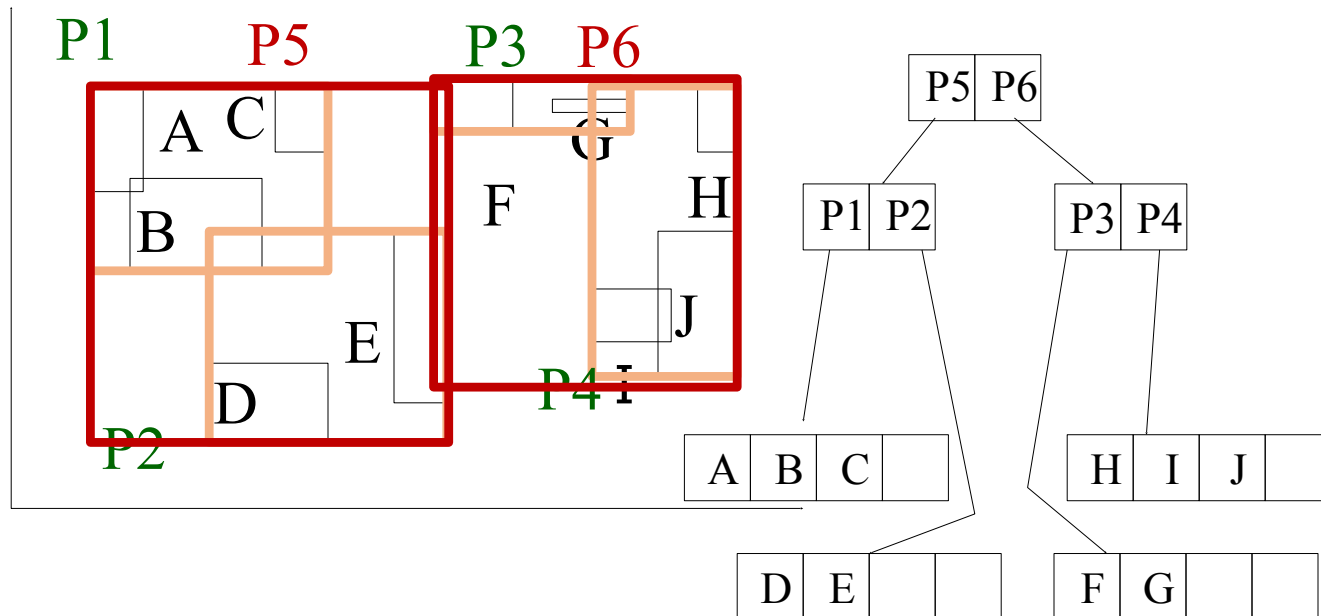


R-tree

- A multi-way external memory tree
- Keys: n-dimensional rectangles, (2 points)
- Index nodes and data (leaf) nodes
- All leaf nodes appear on the same level
 - Leaf node index entries: (l, tuple_id)
 - Non-leaf node entry: (l, child_ptr)
- Every node contains between m and M entries
 - $m \leq M/2$ is the minimum entries per node.
- The root node has at least 2 entries (children)

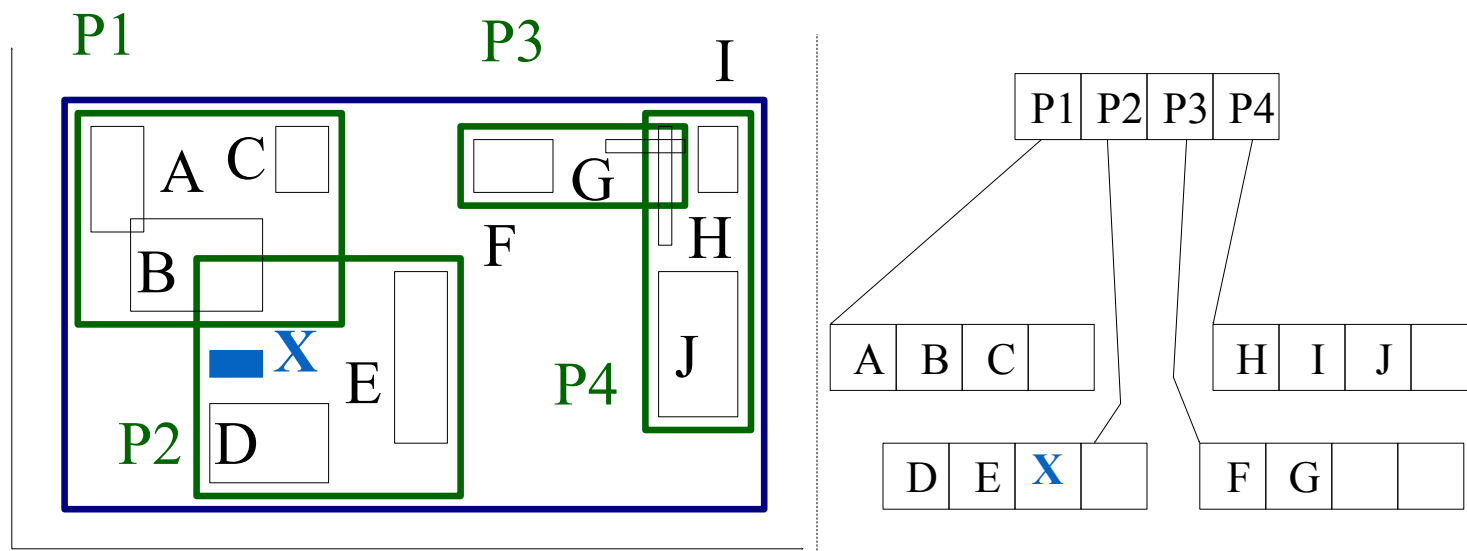
Example

- R trees grow like B+ trees
 - Bottom up
 - eg., w/ fanout 4: group nearby rectangles to parent MBRs; each group -> disk page



R-trees: Insertion

- Insert X: Start from the leaves. Which one?
 - Start at root
 - Go down the tree by choosing child whose rectangle needs **the least enlargement** to include X (Δ area or perimeter...) In case of a tie, choose child with smallest area
 - Least enlargement: increase in **area or perimeter**...a choice!

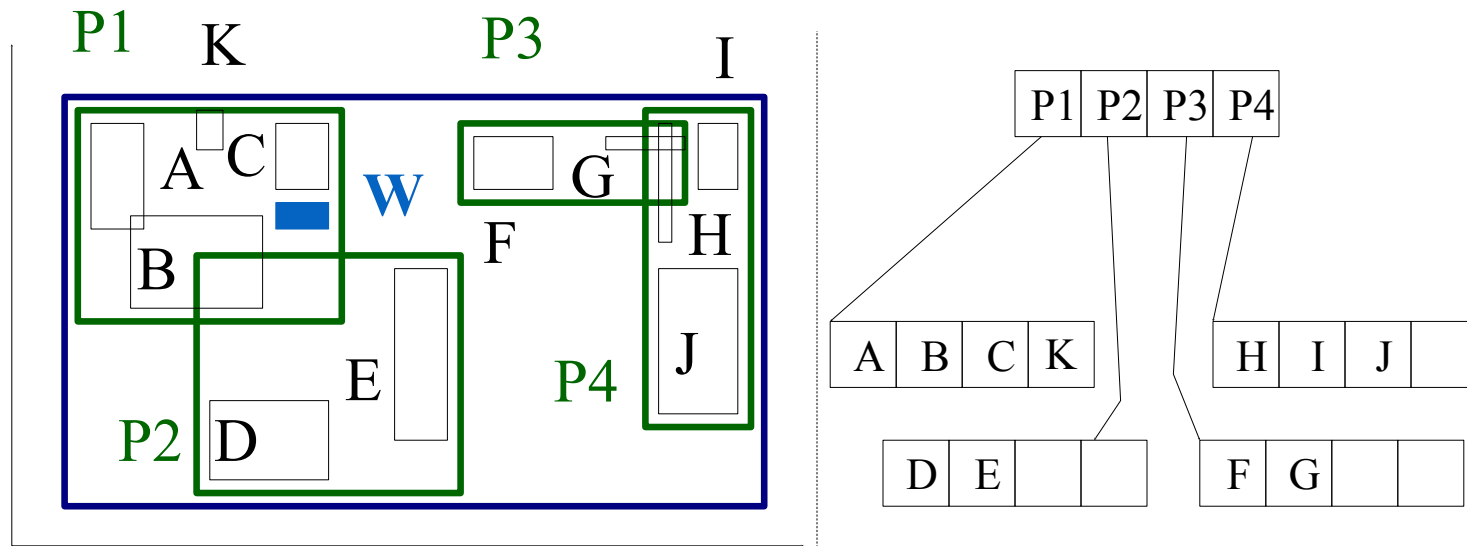


R-trees: Insertion

- How to find the next node to insert a new object Y?
 - Using ChooseLeaf: Find the entry that needs the least enlargement to include Y. Resolve ties using the area (smallest)
- Enlargement measured by change in perimeter of MBR or change in area
- Problem: Can saturate a leaf. In this case, need to ***split***
 - When you split, you readjust MBR in parent to correspond to remaining objects in each of the new nodes.
 - May need to recursively split parent...

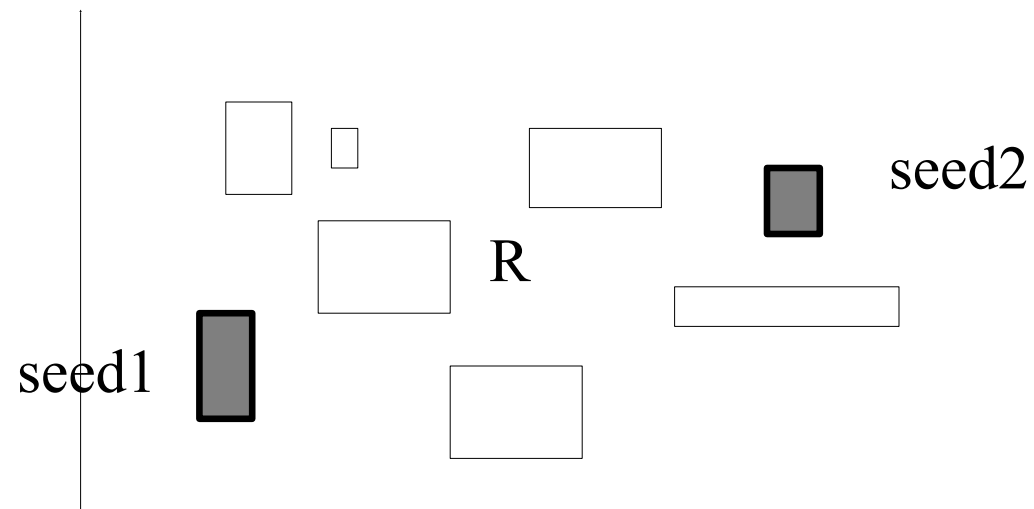
R-trees: Insertion

- If node is full then Split : ex. Insert w



R-trees:Split

- Pick two rectangles as 'seeds' for group 1 and group 2
 - Farthest apart in dimension relative to total spread in dimension



R-Trees: Deletion

- Find the leaf node that contains the entry E
- Remove E from this node
- If underflow:
 - Eliminate the node by removing the node entries and the parent entry
 - Reinsert the orphaned (other entries) into the tree using **Insert**
- Why reinsert?
 - Nodes can be merged with sibling whose area will increase the least, or entries can be redistributed.
 - Reinsertion is easier to implement.